

Contrastive Language-Image Pre-training (CLIP)

Software Engineering
& Data Science
SEIS 766 Vision AI
Dr. Chih Lai

766 Topics from Different Angles

- Object Identification & more
- AutoEncoder (AE)
- Variational AE / VAE
- GAN-1: GAN, C-GAN, W-GAN
- GAN-2: Cycle-GAN, UNIT-GAN
- (Stable) Diffusion, Style Xfer, DeepDream
- CLIP / Classification

Grouping by Characteristics

- Object Identification & more
- AutoEncoder (AE), Variational AE (VAE)
- CLIP / Classification
- GAN-2: Cycle-GAN, UNIT-GAN
- Style Xfer
- GAN-1: GAN, C-GAN, W-GAN
- (Stable) Diffusion, DeepDream

Applicability

- Basic image processing, colors

Discussion Order

- Object Identification & more
- AutoEncoder (AE)
- Variational AE / VAE
- **CLIP** (**C**ontrastive **L**anguage-**I**mage **P**re-training)
- GAN + **Quality** measures
- Cycle-GAN, UNIT-GAN
- C-GAN, W-GAN
- Style Xfer
- (Stable) Diffusion, DeepDream
- DNN for Image Processing
- Morphological Ops in traditional Image Processing



Outline

- **✓** Contrastive *Language-Image Pre-training* (**CLIP**)
- From problem to solution: training steps
- From problem to solution: more on prediction step
- Experiments
- Contrastive loss, programs and pseudo code

CLIP (*Contrastive* Language-Image Pretraining), **OpenAI 2021**

SimCLR (Simple Framework for *Contrastive Learning* of Visual Representations), **Google 2020**

BYOL (Bootstrap Your Own Latent), **Google 2021**

DINOv2 (Self-Distillation with No Labels v2), **Meta AI 2023**

We know AI can do many things... How about???

- Current AI can accurately classify 1000+ objects / classes *individually*



- But, how about *classifying* images **A, B, C?**
describing

- Current AI??

- Cannot / difficult recognize combinations of known objects
- i.e. **NO** compositional generalization
- How about unknown object?



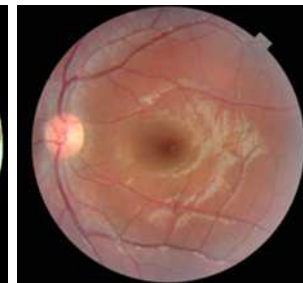
What Are Lacking for Ordinary AI???

What are activities?? Describe images?? Image captions??



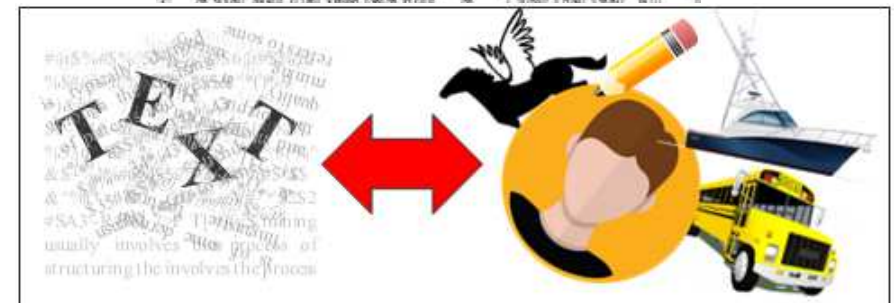
- Show me images with people sleeping in a meeting in late / early hours...
- Show me images w/ dog & cat.....
- Show me x-ray w/ disease A & B &...

What diseases??



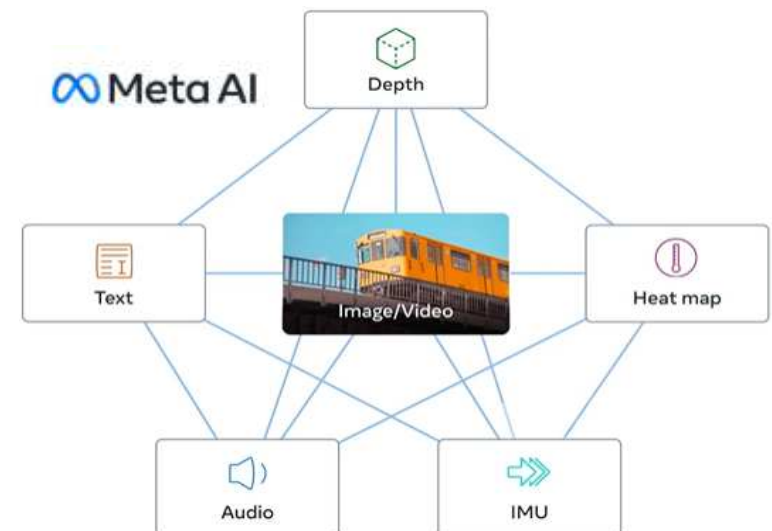
- **1 class prediction** from 1 model for 1 info type
- We need *multi-modal model*
- We need **0-shot** w/o keep re-training
 - on new data of unseen classes of different types

1	232	11.0819	40.0434	25.5789	0.00907	0.60025	-0.1137	200	1	1.0117	0.00016	0.570125	76.591
2	79	29.4915	270.3973	15.5372	6.40749	0.00704	-0.00000	75	1	9.94007	0.050024	0.000131	59.217
3	164	10.1041	50.0414	27.7349	0.162085	0.245634	0.01749	164	1	1.01467	0.703441	0.526302	80.111



Contrastive *L*anguage-*I*mage *P*re-training (**CLIP**) from OpenAI

- **0-shot classification** can apply knowledge stored in a model to unseen scenarios
 - To address: a model works well in a domain of *known* classes, fails in *unseen* others
 - Allows a model to classify data in **unseen combinations of *known* objects** w/o re-training
- **Multi-Modal Model**
 - Train NN on exist ^{any} (image, text) pairs, for classifying new image w/ multiple possible text
 - Unite image and text modalities within a *shared* embedding (**Z**) space
 - Encode text & image as **Zs**, THEN **project** (??) them into a common **embedding (Z)** space
 - So NNs must learn good **Zs** to capture **associations** between images & text (or numeric)
- Example systems:
 - OpenAI CLIP
 - Netflix trained CLIP to combine video & text into a common space for video search
<https://netflixtechblog.com/building-in-video-search-936766f0017c>
 - Meta **ImageBind**
 - CLAP BLIP



OpenAI CLIP, Classify New Image w/ Multiple Possible Text

guacamole (90.1%) Ranked 1 out of 101 labels



✓ a photo of **guacamole**, a type of food.

✗ a photo of **ceviche**, a type of food.

✗ a photo of **edamame**, a type of food.

✗ a photo of **tuna tartare**, a type of food.

✗ a photo of **hummus**, a type of food.

■ **Blue line** = predict probability

■ ✓ = true label

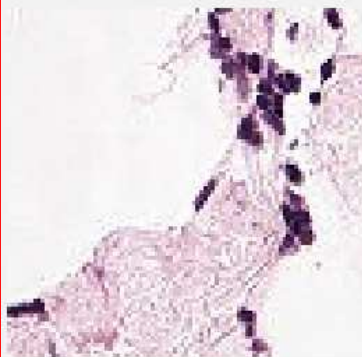
<https://openai.com/research/clip>

Images to Text

multiple diseases??



healthy lymph node tissue (77.2%) Ranked 2 out of 2 labels



✗ this is a photo of **lymph node tumor tissue**

✓ this is a photo of **healthy lymph node tissue**



television studio (90.2%) Ranked 1 out of 397 labels



✓ a photo of a **television studio**.

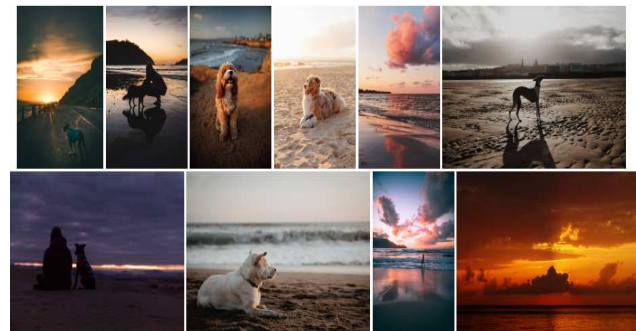
✗ a photo of a **podium indoor**.

✗ a photo of a **conference room**.

✗ a photo of a **lecture room**.

✗ a photo of a **control room**.

sunset clouds at beach with a dog



<https://huggingface.co/spaces/vivien/clip>

Text to Images

Food Waster Application, Images → Text, Text → Images

Why not just compare cosine similarity among all image Zs??



Text Query Enter **Food Waste**

Select Image

Quit

Image 62, Vegetable Scraps, Wilted Lettuce, Chopped Vegetables, Leftover Condiments

Image 36, Salad Components: Leafy greens, Salad Components: Bell pepper, Pasta Dish. CosSim = 0.96164

Image 185, meat, vegetables, sauces. CosSim = 0.95465

Image 3, radishes, leafy greens, vibrant sauce. CosSim = 0.94071

Query
Image
62



Outline

- Contrastive *L*anguage-*I*mage *P*re-training (**CLIP**)
-  From problem to solution: training steps
- From problem to solution: more on prediction step
- Experiments
- Contrastive loss, programs and pseudo code

CLIP on KerasHub vs. Implement Your Own CLIP Loss

CLIP	
https://keras.io/keras_hub/api/models/clip/	
CLIPTokenizer	- CLIPTokenizer class - from_preset method
CLIPImageConverter	- CLIPImageConverter class - from_preset method
CLIPBackbone model	- CLIPBackbone class - from_preset method
CLIPPreprocessor	- CLIPPreprocessor class - from_preset method - tokenizer property

VAI_Slide_CLIP_KerasHub_FAIL !!!!!.ipynb

Load the Model and Preprocessor

```
# Choose a preset, e.g., 'clip_vit_base_patch32'
PRESET = "clip_vit_base_patch32"

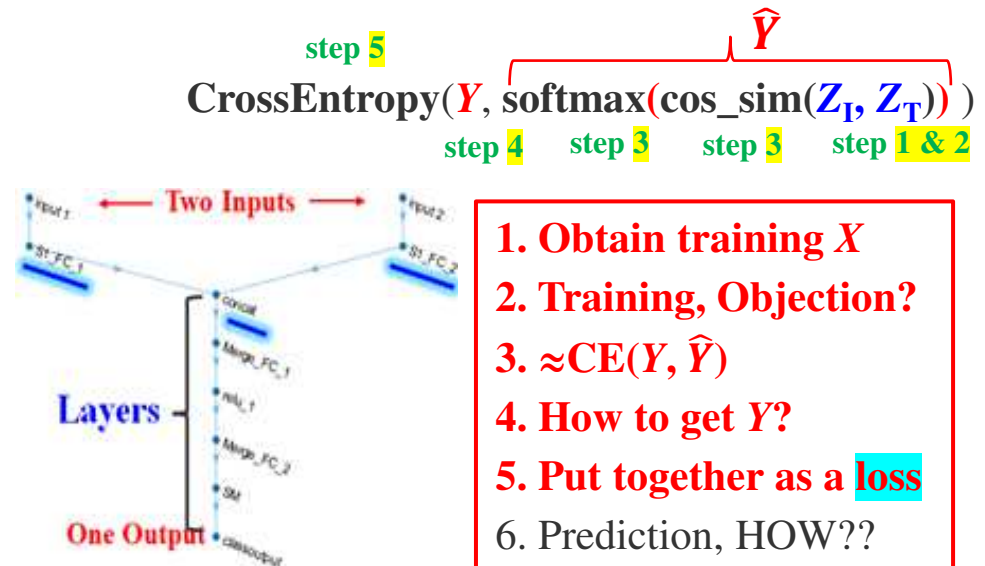
# 1. Load the pre-trained CLIP model (the dual encoder)
# Load the model architecture without pre-trained weights to avoid
clip_model = CLIPBackbone.from_preset(PRESET, load_weights=False)

# 2. Load the preprocessor which handles image and text formatting
preprocessor = CLIPPreprocessor.from_preset(PRESET)

-----
HTTPError                                Traceback (most recent call)
/usr/local/lib/python3.12/dist-packages/kagglehub/exceptions.py in
    65     try:
--> 66         response.raise_for_status()
    67     except requests.HTTPError as e:
```

- All program sections work..
- Except unable to load any pre-trained CLIP
- Also fail with other models from *HuggingFace*

- Instead, just implement your own loss function
- You can then apply CLIP to whatever data



```
def compute_loss(image_embeddings, caption_embeddings, temperature):
    # Normalize embeddings with L2 normalization function from tensorflow
    image_embeddings = tf.nn.l2_normalize(image_embeddings, axis=-1)
    caption_embeddings = tf.nn.l2_normalize(caption_embeddings, axis=-1)

    # logits[i][j] is the dot_similarity(caption_i, image_j)
    logits = softmax(
        tf.matmul(caption_embeddings, image_embeddings, transpose_b=True)
    )

    # images_similarity[i][j] is the dot_similarity(image_i, image_j)
    images_similarity = tf.matmul(
        image_embeddings, image_embeddings, transpose_b=True
    )

    # captions_similarity[i][j] is the dot_similarity(caption_i, caption_j)
    captions_similarity = tf.matmul(
        caption_embeddings, caption_embeddings, transpose_b=True
    )

    # targets[i][j] = average dot_similarity(caption_i, caption_j) and dot_similarity(image_i, image_j)
    targets = softmax(
        (captions_similarity + images_similarity) / 2 * temperature
    )

    # Compute the loss for the captions using crossentropy
    captions_loss = categorical_crossentropy(
        y_true=captions_targets, y_pred=tf.transpose(logits), from_logits=False
    )

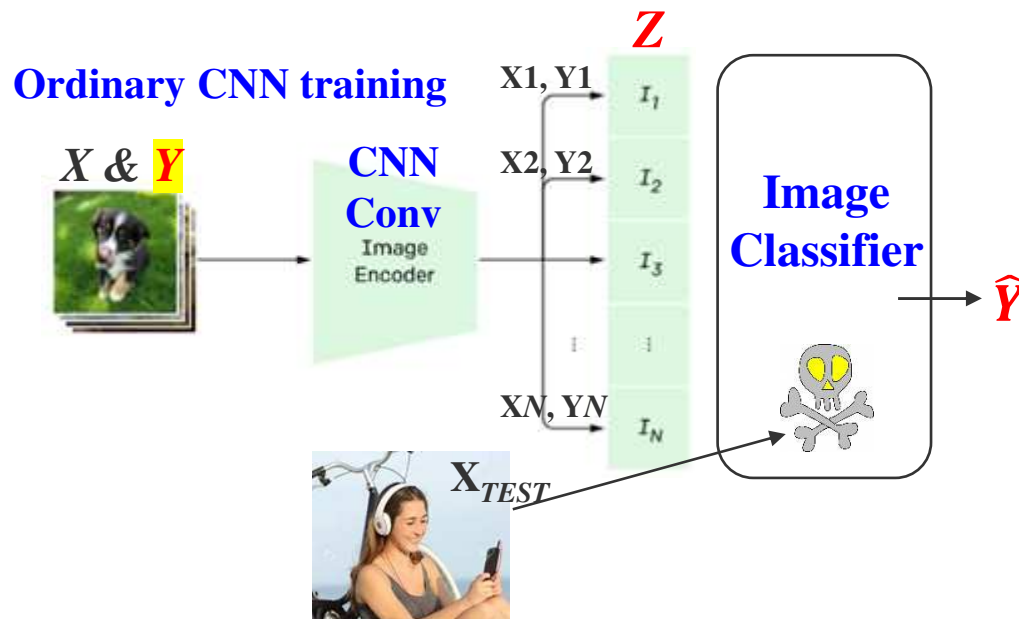
    # Compute the loss for the images using crossentropy
    images_loss = categorical_crossentropy(
        y_true=images_targets, y_pred=tf.transpose(logits), from_logits=False
    )

    # Return the mean of the loss over the batch
    return (captions_loss + images_loss) / 2
```

~ 30 lines

Let's Begin with Problem, Then Solution

- Training CNN with (X / images, Y / classes)
- Expensive to get Y for each X / image
- Testing images with unknown combinations will end up ??????
- Solution??
 - Keep re-training the model w/ instances of new scenarios / new combined Y s???

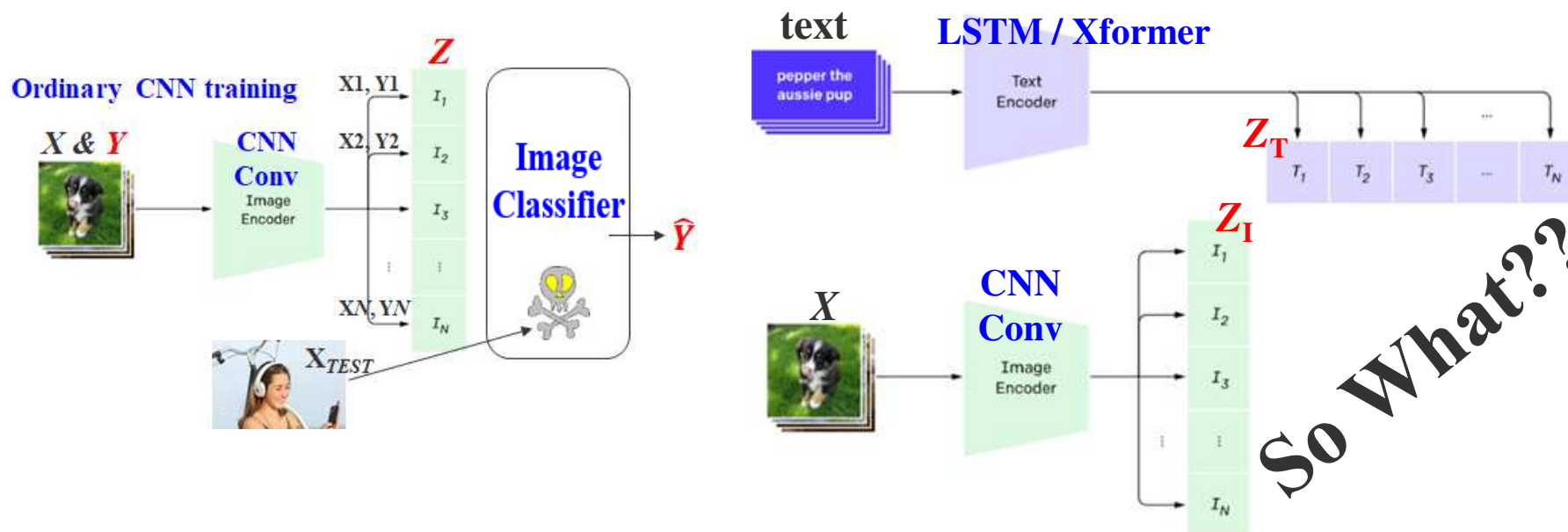


1. Obtain training X , where's Y ??
2. Training, Objection? Y ?
3. $\approx CE(Y, \hat{Y})$, where's $\hat{Y}$?
4. How to get Y ?
5. Put together = **Contrastive Loss**
6. Prediction, HOW??

Step 1, Collecting Data

- Training CNN with (X / images, Y / classes)
- Expensive to get Y for each X / image
- Testing images with unknown combinations will end up **??????** Solution??
- Instead (X , Y) pairs, find X pairs = (image, its text description) on WWW
- **Without** Y So how do we do training??
- Train **2** separated NNs to get Z_I & Z_T and associate them...
- BUT, we need an objection (??) to train...

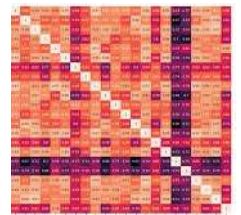
1. Obtain training X , where's Y ??
2. Training, Objection? Y ?
3. $\approx CE(Y, \hat{Y})$, where's $\hat{Y}$?
4. How to get Y ?
5. Put together = *Contrastive Loss*
6. Prediction, HOW??



1. Obtain training X , where's Y ??
2. Training, Objection? Y ?
3. $\approx \text{CE}(Y, \hat{Y})$, where's $\hat{Y}$?
4. How to get Y ?
5. Put together = *Contrastive Loss*
6. Prediction, HOW??

Step 2, Training Similarity for Achieving WHAT???

- Instead of (X, Y) pairs, find $X = (\text{images}, \text{text})$ on WWW
 - Get Z s of Images & Text as Z_I & Z_T , then compute their **cosine similarity**
 - We know each cosine similarity is $[-1 : 1]$, so what??
 - **Objection:** train 2 NNs to produce Z_I & Z_T that has *good cosine similarity*...
 - “good cosine similarity”?? What does that mean? **HOW?**
- ➔ Such that sim-matrix has all **1**s (or as **↑** as possible) on diagonal, i.e. (img, txt) pairs
- **HOW???**

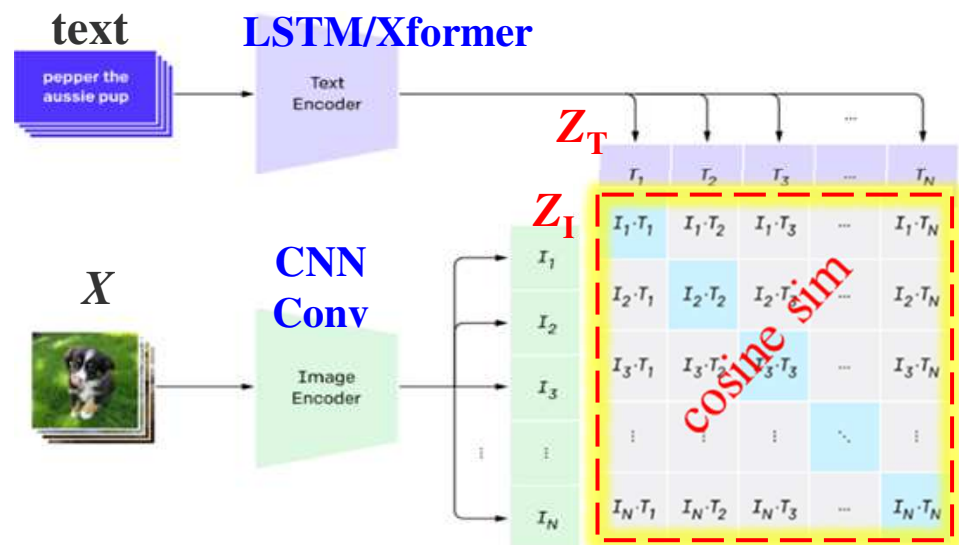


cosineSimilarity(Z_I, Z_T)

from sklearn.metrics.pairwise import cosine_similarity

cosine_similarity(Z_I, Z_T) **### OR...**

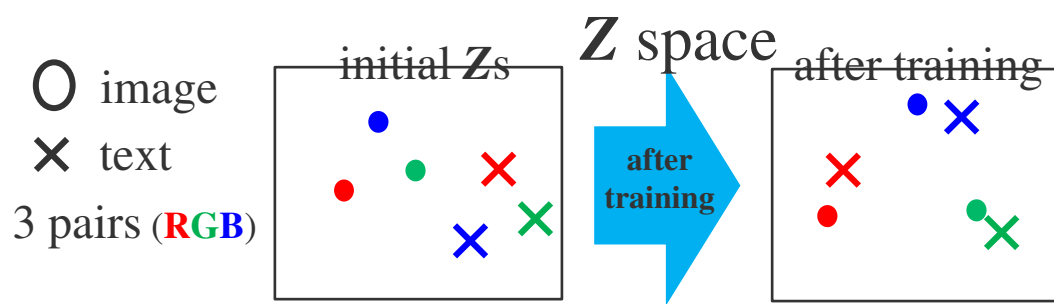
tf.**matmul**(Z_I, Z_T , transpose_b=True)



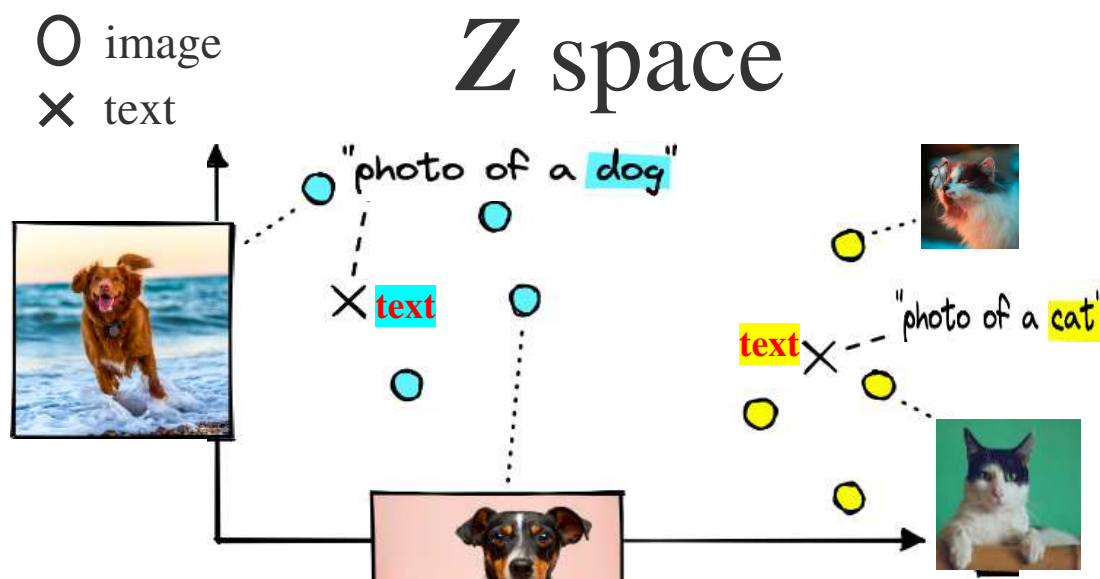
Step 2, Learning Objection = **Contrastive Loss**



- Instance pairs from WWW must be **closer** in **Z** space... & dissimilar pairs **apart**
 - Aim to create a joint embedding space where visual & textual **Zs** are aligned
 - Allow **multi-modal model** *associations* (or *association* among any type of inputs)



training so Zs of similar images & text are close, i.e. (img, txt) pairs are close

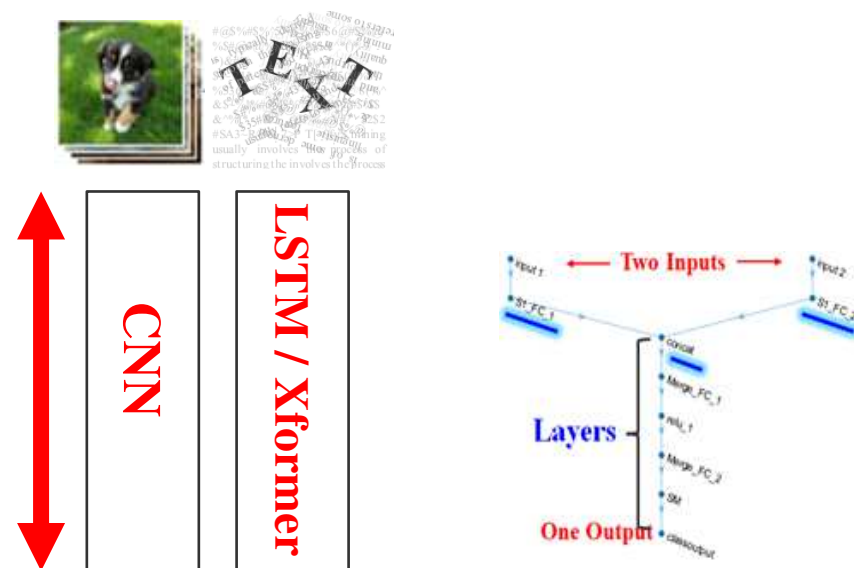
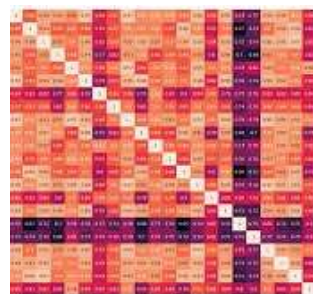
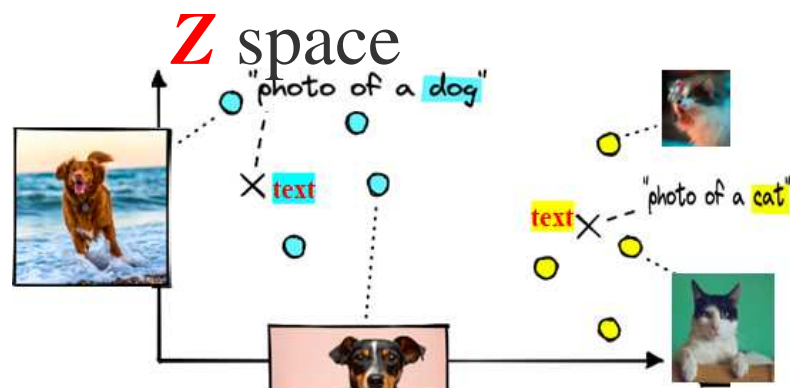
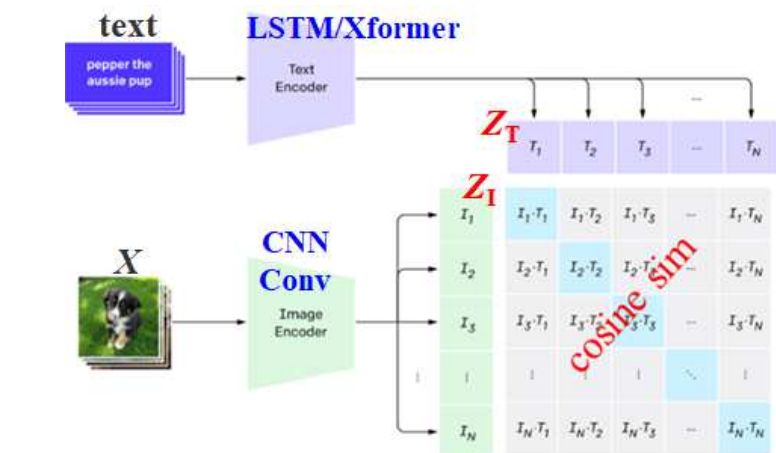


		text Z_T				
		T_1	T_2	T_3	...	T_N
image Z_I	I_1	$I_1 \cdot T_1$	$I_1 \cdot T_2$	$I_1 \cdot T_3$	...	$I_1 \cdot T_N$
	I_2	$I_2 \cdot T_1$	$I_2 \cdot T_2$	$I_2 \cdot T_3$	...	$I_2 \cdot T_N$
	I_3	$I_3 \cdot T_1$	$I_3 \cdot T_2$	$I_3 \cdot T_3$	...	$I_3 \cdot T_N$
	...	...	...	...	...	...
	I_N	$I_N \cdot T_1$	$I_N \cdot T_2$	$I_N \cdot T_3$	...	$I_N \cdot T_N$

<https://www.pinecone.io/learn/series/image-search/zero-shot-object-detection-clip/>

HOW??

Closer Look of CLIP Architecture



$$Z_I$$

$$Z_T$$

$I_1 \cdot T_1$	$I_1 \cdot T_2$	$I_1 \cdot T_3$	$-$	$I_1 \cdot T_N$
$I_2 \cdot T_1$	$I_2 \cdot T_2$	$I_2 \cdot T_3$	$-$	$I_2 \cdot T_N$
$I_3 \cdot T_1$	$I_3 \cdot T_2$	$I_3 \cdot T_3$	$-$	$I_3 \cdot T_N$
$I_N \cdot T_1$	$I_N \cdot T_2$	$I_N \cdot T_3$	$-$	$I_N \cdot T_N$

cosine sim

1. Obtain training X , where's Y ?
2. Training, Objection? Y ?
3. $\approx CE(Y, \hat{Y})$, where's $\hat{Y}$?
4. How to get Y ?
5. Put together = *Contrastive Loss*
6. Prediction, HOW??

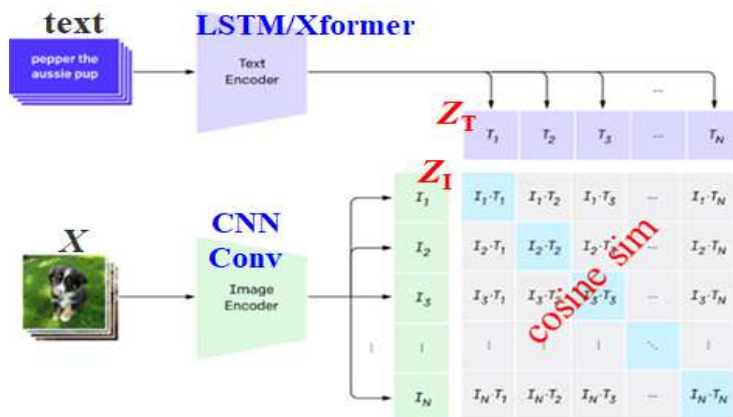
Step 3, Compute 2 Predicted Logits or Probs. $\hat{Y}$ s Matrices

1. Obtain training X , where's Y ??
2. Training, Objection? Y ?
3. $\approx \text{CE}(Y, \hat{Y})$, where's $\hat{Y}$?
4. How to get Y ?
5. Put together = *Contrastive Loss*
6. Prediction, HOW??

- **Step 2:** Compute cosine similarity from Z s of Images and Text as $-1 \leq \text{CosSim} \leq +1$
- **Step 2:** Train 2 NNs to gen Z s s.t. sim-matrix has as $\uparrow$ as possible on **diagonal**, HOW??

- Train **each** of 2 NNs on **contrastive loss** based on **cross entropy (C.E.)**
- **CE** needs predictions $\hat{Y}$ s (as probabilities) that sum = 1 on columns... and rows
 - But, during training $\Sigma \text{CosSim}_{\text{col}} \neq 1$ & $\Sigma \text{CosSim}_{\text{row}} \neq 1$ **Solution??**
 - **softmax** on each **row** (img-txt) = **probs.** matrix $\hat{Y}_1 = \text{sim}$ of Z_I, Z_T
 - **softmax** on each **column** (txt-img) = **probs.** matrix $\hat{Y}_2 = \text{sim}$ of Z_T, Z_I

in fact, $\hat{Y}_2 = \hat{Y}_1'$



img-to-txt
txt-to-img

$\hat{Y}$

Cross Entropy =

$$\frac{-1}{m} \sum_{i=1}^m Y_i \log(\hat{Y}_i)$$

predicted probs. $\hat{Y}$ s

true targets Y

$$\begin{aligned} \hat{Y} &= \text{softmax}(\text{cos_sim}(Z_I, Z_T)) \\ &= \text{probs.} = \text{norm sim between } Z_I, Z_T \end{aligned}$$

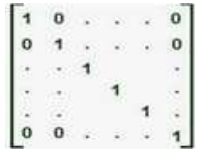
$$\text{softmax}(\text{cosineSimilarity}(Z_I, Z_T))$$

- How do we get one true target Y sim-matrix for cross entropy??

Step 4.1, Get 1 Y Similarity Matrix from Positive Pairs 4. How to get Y?

■ How to get 1 true target Y? *hard* or *soft*

● *HARD* = *Target Y* = identity matrix i.e. diagonals = 1, else = 0 (CLIP)



● *Soft* = *Target Y* = $\text{softmax}(\text{img-img similarity} + \text{txt-txt similarity}) / 2$

• Any *positive pair* of (img, img), (txt, txt), or (**img**, **txt**) must be similar to themselves

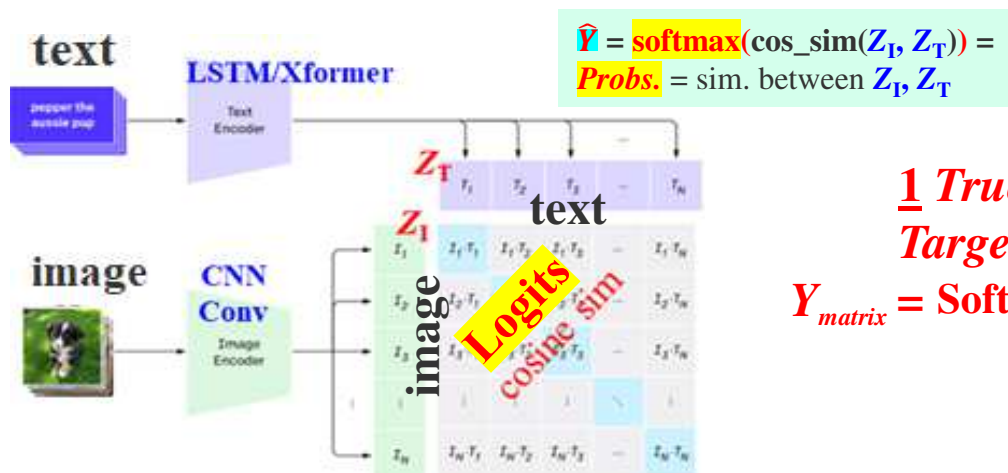


• for *semantic overlap*, regularize over-confidence hard noise, smooth **Z** vs. fragmented **Z**

$$\text{CE}(\underbrace{Y}_{\text{step 4}}, \underbrace{\text{softmax}(\cos_sim(\underbrace{Z_I}_{\text{step 4}}, \underbrace{Z_T}_{\text{step 3}}))}_{\text{step 3}}))$$

$$\text{Cross Entropy} = \frac{-1}{m} \sum_{i=1}^m Y_i \log(\hat{Y}_i)$$

predicted probs. $\hat{Y}_i$
true targets Y_i



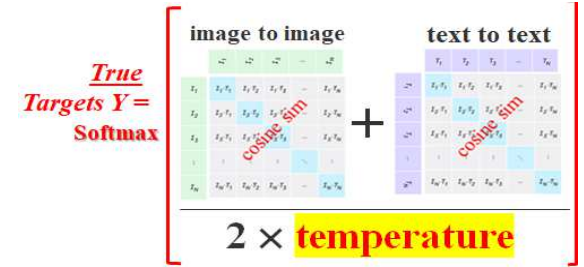
1 True
Target
 $Y_{matrix} = \text{Softmax}$

$$\left[\begin{array}{c} Z_I \text{ img-to-img} \\ Z_T \text{ txt-to-txt} \end{array} \right] + \left[\begin{array}{c} \text{cosine sim} \\ \text{cosine sim} \end{array} \right]$$

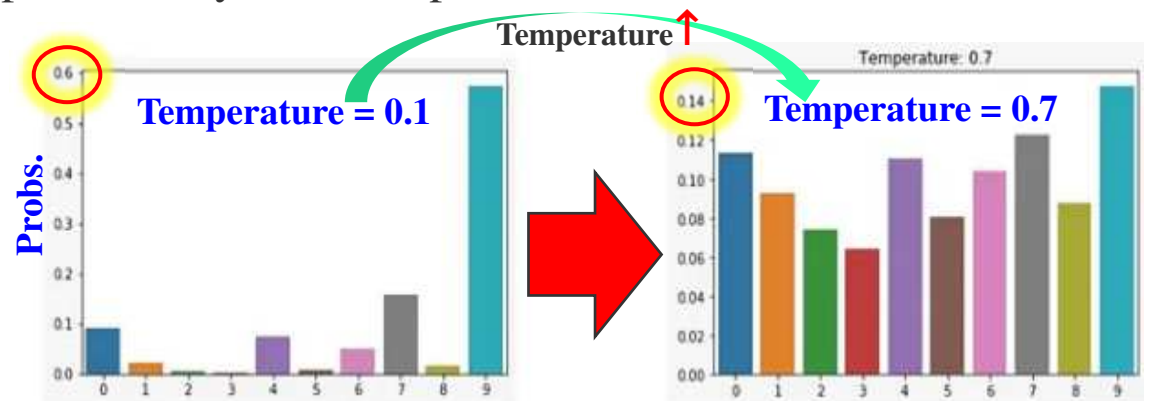
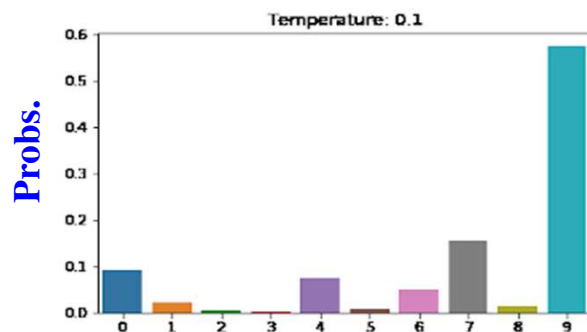
$2 \times \text{temperature???$

Add **T**emperature in Softmax

ChatGPT



- **Softmax** $\frac{e^{W^T X_i}}{\sum_i^m e^{W^T X_i}} = \frac{e^a}{\text{sum}(e^a)} \frac{e^{a/T}}{\text{sum}(e^{a/T})}$
- Before applying softmax, divide each **logit $\hat{y}$** & **target y** by a temperature (**T**)
- ↑temperature (i.e. **T** > 1):
 - **Uniform** lower prob. distrib., **decrease** contrast between + (similar) and – pairs (dissimilar)
 - Learn more generalized (open) associations between Z_I & Z_T , i.e. more forgiving to dissimilar pairs
 - Less certain which pairs be associated, so more **diverse** or **creative** (unexpected) outputs
- ↓temperature (i.e. **T** < 1):
 - **Sharpen** prob. distrib., **increase** contrast between + and – pairs, but, sensitive to small Δ
 - Learn more **discriminative** (close) associations between Z_I & Z_T
 - More certain, **less diverse** output and may lead to repetitions (**mode collapse, overfit to a small groups**)



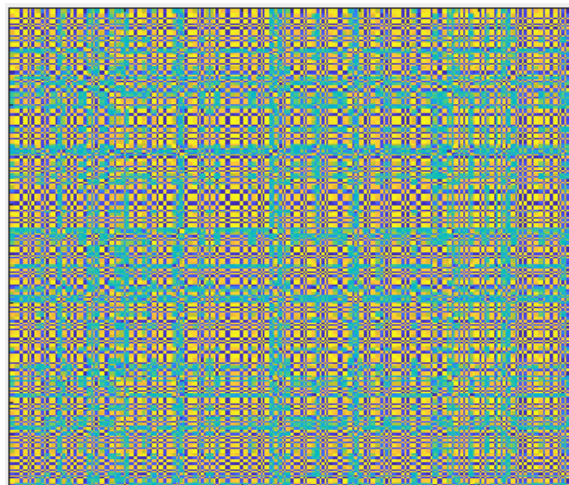
Temperature Impact to Similarity Matrix

- ↑temperature (i.e. $T > 1$):
 - **Uniform** lower prob. distrib., **decrease** contrast between + (similar) and – pairs (dissimilar)
 - Learn more generalized (open) associations between Z_I & Z_T , i.e. more forgiving to dissimilar pairs
 - Less certain which pairs be associated, so more *diverse* or *creative* (unexpected) outputs
- ↓temperature (i.e. $T < 1$):
 - **Sharpen** prob. distrib., **increase** contrast between + and – pairs, but, sensitive to small Δ
 - Learn more **discriminative** (close) associations between Z_I & Z_T
 - More certain, *less diverse* output and may lead to repetitions (**mode collapse, overfit to a small groups**)

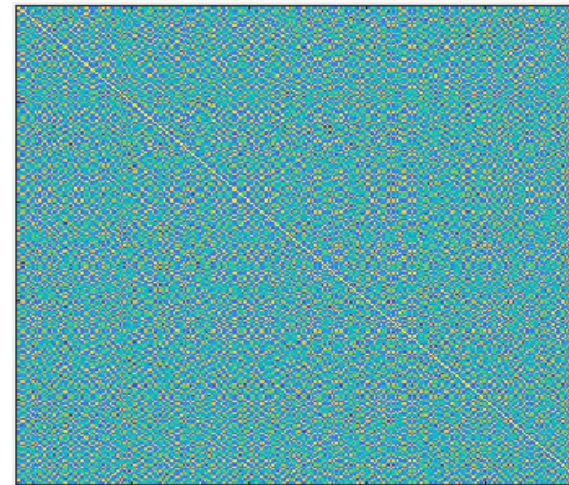


TEXT

$T = 1.5$



$T = 0.7$



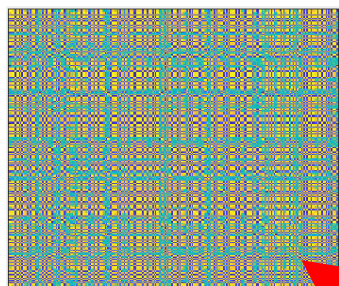
Diagonal Values over Epochs

- $T \uparrow \rightarrow \text{input to softmax} \downarrow \rightarrow P_{i,i} \downarrow$
 $\rightarrow$ less certain which pair is associated

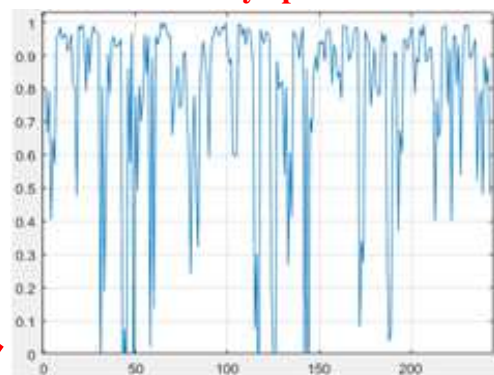
$$\text{True Targets } Y = \text{Softmax} \left[\frac{\text{image to image} + \text{text to text}}{2 \times \text{temperature}} \right]$$

The diagram shows two matrices being added and then divided by $2 \times \text{temperature}$. The first matrix, labeled 'image to image', has a diagonal of green cells and off-diagonal cells of various colors. The second matrix, labeled 'text to text', has a diagonal of purple cells and off-diagonal cells of various colors. The result is a matrix where the diagonal is a mix of green and purple, and off-diagonal cells are also present.

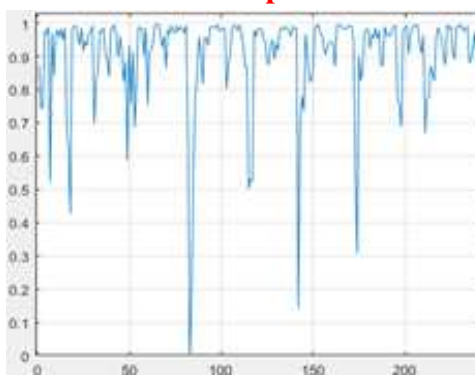
T= 1.5



Diagonal Values
early epoch



middle epoch

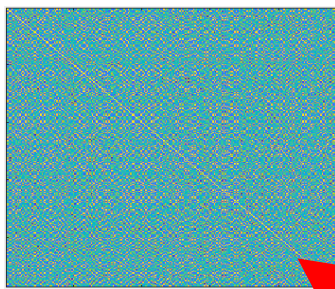


Diagonal Values
later epoch

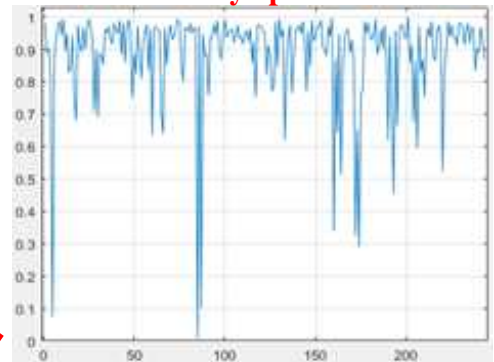


open associations

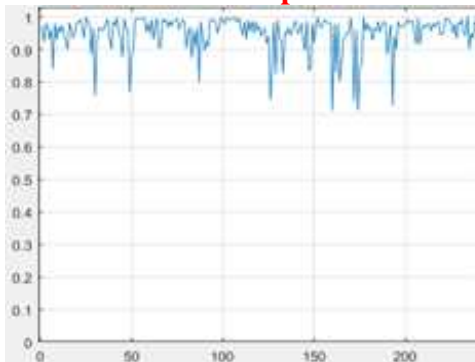
T= 0.7



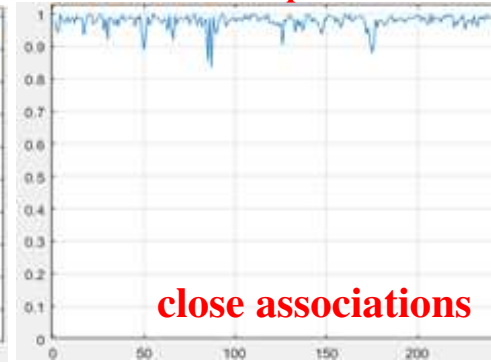
Diagonal Values
early epoch



middle epoch



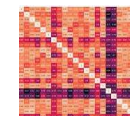
Diagonal Values
later epoch



close associations

Step 4.2, Final CE from **Y** Self Similarity Matrix

4. How to get **Y**?



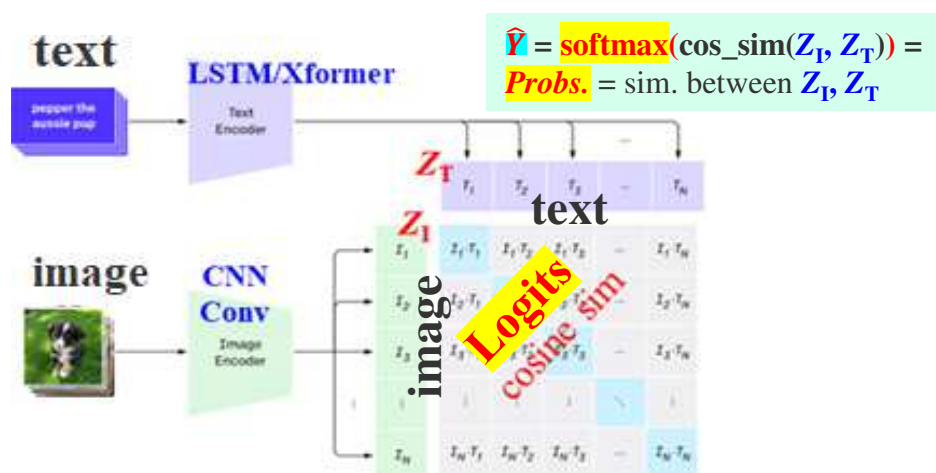
- How to get **1 target Y**? **target** matrix must have **diagonals = 1**
 - Soft Target Y** = $\text{softmax}(\text{img-img similarity} + \text{txt-txt similarity}) / 2$
 - Each **positive pair** = input (img, img), (txt, txt), (**img**, **txt**) pairs must be similar to themselves
- Next, compute **2 contrastive losses** between **1 Y** and **2 $\hat{Y}$ s** (**1 target** & **2 logits**)
 - $CE_1(Y, \hat{Y}_1 = \text{img-txt})$, $CE_2(Y, \hat{Y}_2 = \text{txt-img})$ in fact, $\hat{Y}_2 = \hat{Y}_1'$
 - Use **2 contrastive losses** CE_1 & CE_2 to compute **gradient** & update θ s in **2 NNs**

$$CE(Y, \underbrace{\text{softmax}(\cos_sim(Z_I, Z_T))}_{\hat{Y}})$$

step 4 step 4 step 3 step 3

img-to-txt
txt-to-img
Cross Entropy = $-\frac{1}{m} \sum_{i=1}^m Y_i \log(\hat{Y}_i)$

predicted probs. $\hat{Y}$ s
true targets **Y**



option 1 $\text{softmax}\left(\frac{S_{ii} + S_{tt}}{2 \times T}\right)$

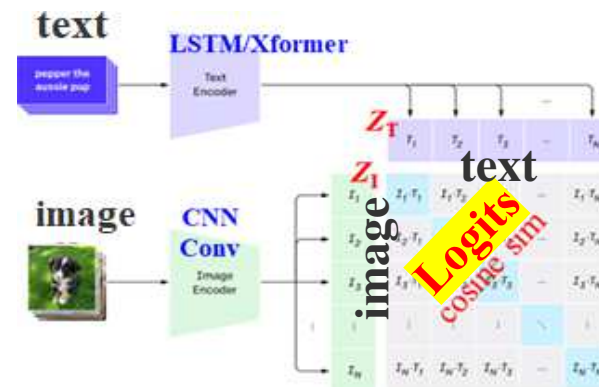
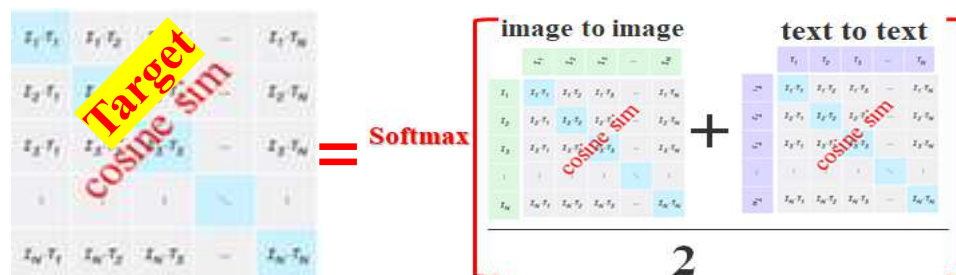
Soft Target Y

✓ option 2 $\frac{\text{softmax}(S_{ii}) + \text{softmax}(S_{tt})}{2 \times T}$

Solution, Step 5, Put Together **Contrastive Loss**

- How to get **1 target Y**? *target* matrix must have diagonals = 1
 - Soft Target Y** = softmax(*img-img similarity* + *txt-txt similarity*) / 2

1. Obtain training X , where's Y ??
2. Training, Objection? Y ?
3. $\approx \text{CE}(Y, \hat{Y})$, where's $\hat{Y}$?
4. How to get Y ?
5. Put together = *Contrastive Loss*
6. Prediction, HOW??



- Finally, compute **2 contrastive losses** between **1 Ys** and **2 $\hat{Y}$ s** (**1 targets** & **2 logits**)

$$1. \text{ text_loss} = \text{CE}(\text{TargetY}, \text{Probs.})$$

$$\text{Probs.} = \text{softmax}(\text{cos_sim}(Z_T, Z_I))$$

$$2. \text{ image_loss} = \text{CE}(\text{TargetY}', \text{Probs.}')$$

$$\text{Probs.}' = \text{softmax}(\text{cos_sim}(Z_I, Z_T))$$

$$\bullet \text{ Contrastive loss} = (\text{text_loss} + \text{image_loss}) / 2$$

$$\text{CE}(Y, \text{softmax}(\text{cos_sim}(Z_I, Z_T)))$$

step 4 step 4 step 3 step 3

$$\text{CE}(\text{target}', \text{probs.}') = \frac{-1}{m} \sum_{i=1}^m Y_i \log(\hat{Y}_i)$$

target', probs.' predicted probs. $\hat{Y}$ s true targets Y

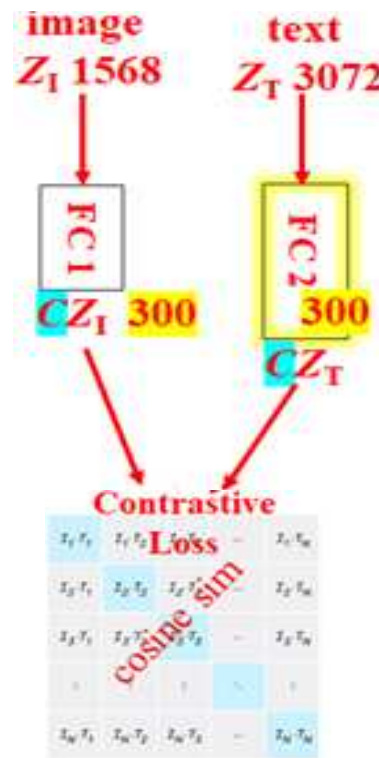
Contrastive Loss

Pseudo Code

```

Define net1, net2 (i.e. FC1, FC2)
T = temperature

Create batch queue mbq
Loop epochs
    shuffle(mbq)
    # get imgZ from CNN if needed
    # get txtZ from LSTM if needed
    TotLoss, gradients =
        Loss(net1, net2, imgZ, txtZ, T)
    update  $\theta$ s in net1 and net2
End
    
```



$$\text{CE}(\overset{\text{step 5}}{Y}, \overset{\text{step 4}}{\underbrace{\text{softmax}(\overset{\text{step 3}}{\text{cos_sim}(\overset{\text{step 3}}{Z_I, Z_T})}}_{\hat{Y}})})$$

```

function Contrastive Loss
    imgCZ = forward(net1, imgZ, ....) # image step 1
    txtCZ = forward(net2, txtZ, ....) # text step 1

    # may need to normalize them for cos_sim
    imgCZ = vec_norm(imgCZ)
    txtCZ = vec_norm(txtCZ)

    # get predicted sim between both CZs
    logits = cosSim(imgCZ, txtCZ) / T # step 3

    imgSim = cosSim(imgCZ, imgCZ)
    txtSim = cosSim(txtCZ, txtCZ) } Y = positive pairs sim (diags = 1)

    # get target sim between both CZs, # step 4 + SM
    targets = softmax( [ (imgSim + txtSim) / (2×T) ] )

    # Inputs = probs. & Output ≥ 0 # step 5 + SM
    imgLoss = crossEntropy(softmax(logits), targets)
    txtLoss = crossEntropy(softmax(logits), targets)

    TotLoss = (imgLoss + txtLoss) / 2
    #
    Compute Gradients of TotLoss w.r.t. net1, net2
    
```

- Each *positive pair* indicates text (or image) should be similar to itself
- Target Y = sums **cosSim** of the *positive* pairs
- C.E.** = predict probs. *against* targets

<https://medium.com/@sthanikamsanthosh1994/contrastive-language-image-pre-training-clip-connecting-text-to-image-d9ef92462eda>

F.Y.I. Contrastive Loss, N images and N texts

1. Obtain training X , where's Y ??
2. Training, Objection? Y ?
3. $\approx \text{CE}(Y, \hat{Y})$, where's $\hat{Y}$?
4. How to get Y ?
5. Put together = *Contrastive Loss*
6. Prediction, HOW??

■ CLIP treats image-text matching as a **classification problem**:

- Given image x_i , NN_f classify which text y_j is best match (i.e. class)
- Similarly, given text y_j , NN_g classify which image x_i is best match (i.e. class)

■ Contrastive Loss with Cross-Entropy

- Similarity matrix $S_{i,j} = \text{cosine_similarity}(f(x_i), g(y_j))$ f and g are 2 NNs

		text Z_T				
		T_1	T_2	T_3	...	T_N
image Z_I	I_1	I_1T_1	I_1T_2	I_1T_3	...	I_1T_N
	I_2	I_2T_1	I_2T_2	I_2T_3	...	I_2T_N
	I_3	I_3T_1	I_3T_2	I_3T_3	...	I_3T_N
	...	...	...	...	...	...
	I_N	I_NT_1	I_NT_2	I_NT_3	...	I_NT_N

For image-to-text classification, we apply softmax over each row of S to get a probability distribution:

$$\text{step 3} \quad \text{1. softmax} \quad P(y_j|x_i) = \frac{\exp(S_{i,j}/\tau)}{\sum_{k=1}^N \exp(S_{i,k}/\tau)} \quad \tau = \text{temperature} \quad \frac{\text{softmax}(S_{ii}) + \text{softmax}(S_{tt})}{2 \times T} \quad \text{option 2}$$

Then, we compute the cross-entropy loss, assuming the correct match is at index i :

$$\text{step 5} \quad \text{2. cross entropy} \quad L_{\text{image}} = - \sum_{i=1}^N \log P(y_i|x_i)$$

$$\text{XEntropy}(\underbrace{Y}_{\hat{Y}}, \text{softmax}(\text{cos_sim}(Z_I, Z_T)))$$

step 5 step 4 step 3 step 3

Similarly, for text-to-image classification, softmax is applied over each column:

$$\text{1. softmax} \quad P(x_j|y_i) = \frac{\exp(S_{i,j}/\tau)}{\sum_{k=1}^N \exp(S_{k,j}/\tau)}$$

$$\text{2. cross entropy} \quad L_{\text{text}} = - \sum_{i=1}^N \log P(x_i|y_i)$$

Final Contrastive Loss

$$L = \frac{1}{2} (L_{\text{image}} + L_{\text{text}})$$

Outline

- Contrastive *L*anguage-*I*mage *P*re-training (**CLIP**)
- From problem to solution: training steps
-  From problem to solution: **now for the prediction step**
- Experiments
- Contrastive loss, programs and pseudo code

CLIP (*Contrastive* Language-Image Pretraining), **OpenAI 2021**

SimCLR (Simple Framework for *Contrastive Learning* of Visual Representations), **Google 2020**

BYOL (Bootstrap Your Own Latent), **Google 2021**

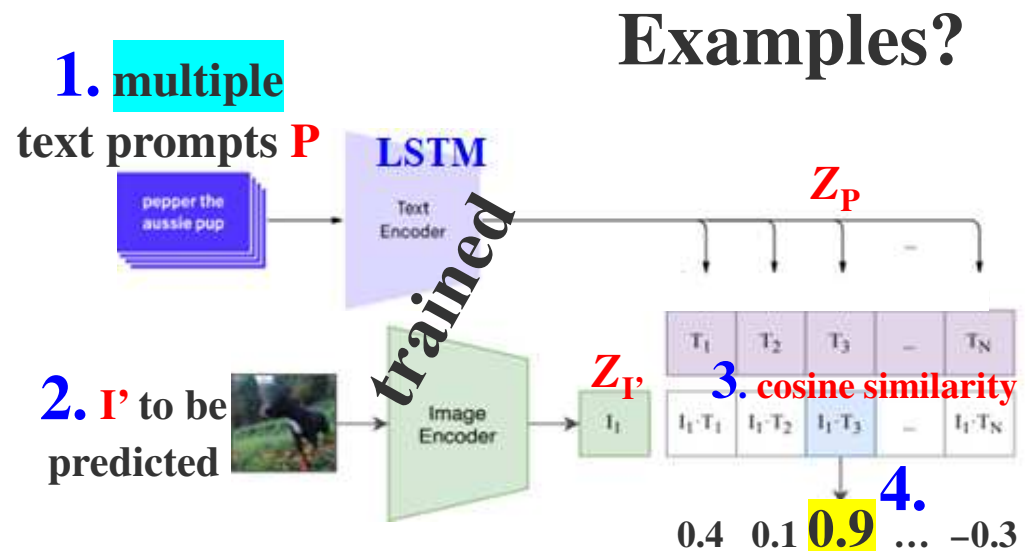
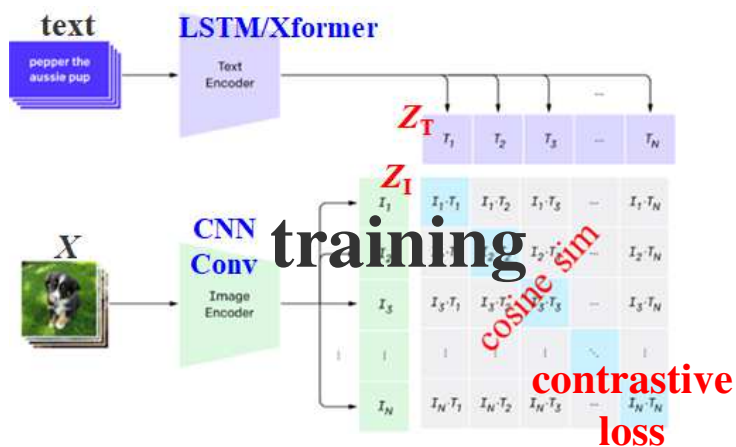
DINOv2 (Self-Distillation with No Labels v2), **Meta AI 2023**

Solution, Step 6, Prediction

- Instead of (X, Y) pairs, find X / image and its text description on WWW
- Compute cosine similarity $[-1 : 1]$ from Z s of Images and Text (Z_I & Z_T)
- Training objection? Train 2 encoders to gen Z s so sim-matrix has $\uparrow$ in **diagonal**

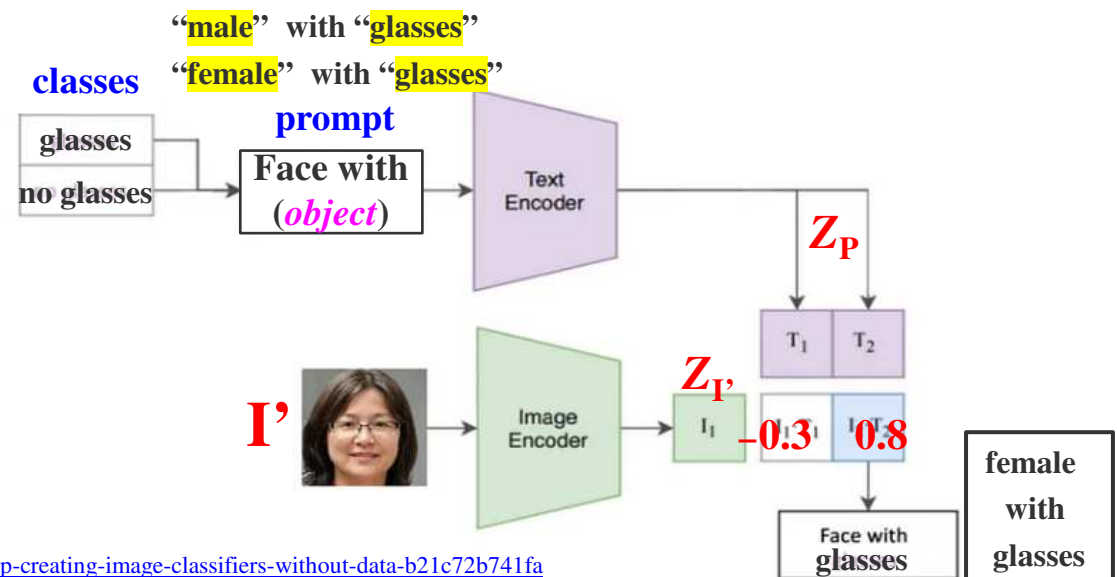
- After training, how do we do **classify** (predict) a new image I' ?? **Method 1**

- ?? 1. Create **multiple** potential text prompts P to input to **trained** LSTM/Xformer to get Z_P
2. Input the image I' to the **trained** CNN to get $Z_{I'}$
3. Compute cosine sim between $Z_{I'}$ & Z_P to see: which Z_P is **highest** (i.e. **closest** to $Z_{I'}$)
4. Classifying I' into the "text prompt" w/ the highest similarity



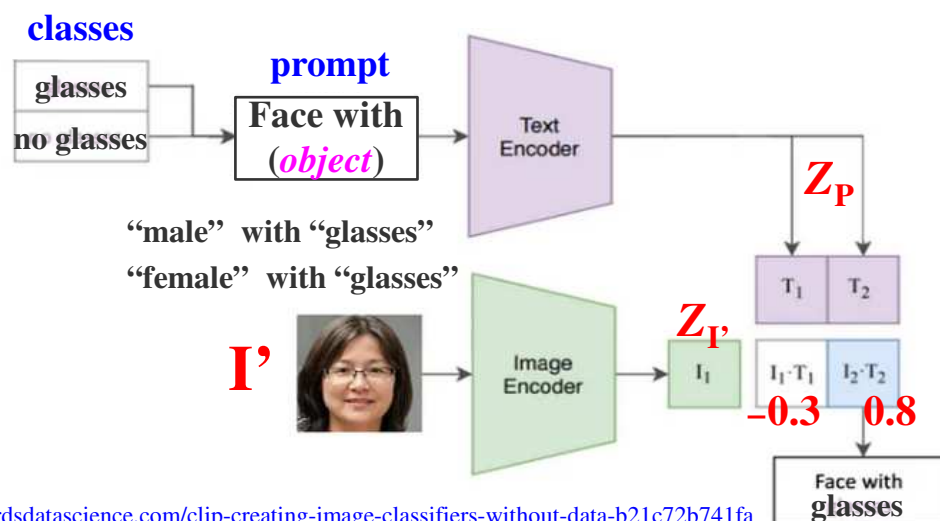
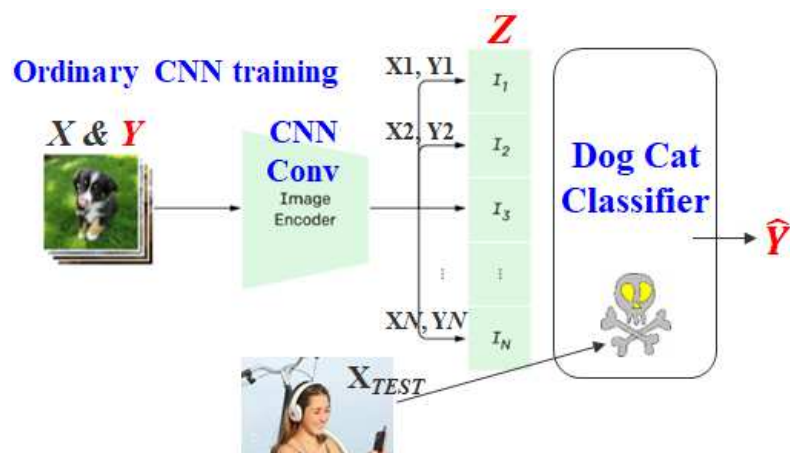
CLIP Prediction, Method 1, More Detail

- Input image I' into CNN to obtain image embedding Z_I
- Create **text prompts** (or even numeric records) P as **potential classes**
 1. Create a text prompt w/ a placeholder “??” (i.e. P = “Face with ??”)
 2. Replace “??” with labels you want to classify, THEN insert to many different prompts
 - i.e. Face with “glasses”, or Face with “no glasses”
- Input all text prompts P into text encoder to obtain their text embeddings Z_P
- Classification = compute **cosine similarity** between text Z_P and image Z_I
 - Output classification by choosing the text prompt w/ highest similarity



Method 1, Where Is the **Zero-Shot** Classification??

- Definition: a model can handle new inputs (images / text) even if it wasn't explicitly trained on those specific examples with the specific label ***Y***
- Allows a model to classify unseen data in **unseen classes** without keep re-training
 - **NO** need to explicitly *re-trained* models with **ALL** the following 4 classes:
 - "*male with glasses*", "*male without glasses*", "*female with glasses*", "*female without glasses*"
- Just need to expose models to large / various pairs of (text, images)

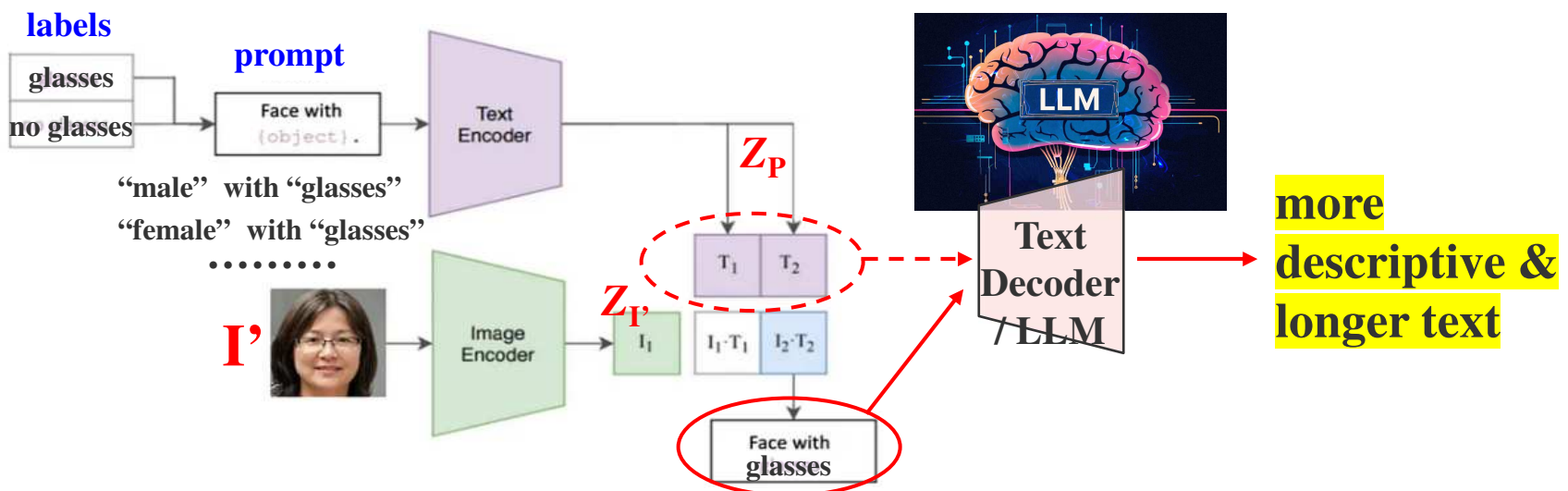


<https://towardsdatascience.com/clip-creating-image-classifiers-without-data-b21c72b741fa>

<https://www.pinecone.io/learn/series/image-search/zero-shot-image-classification-clip/>

Increasing Descriptiveness of Text Output

- Generated text may not be descriptive enough??? Solution???
- How???



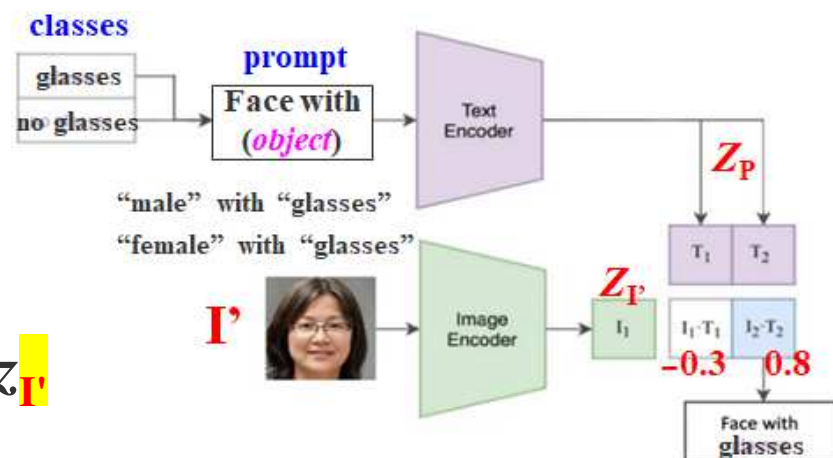
<https://towardsdatascience.com/clip-creating-image-classifiers-without-data-b21c72b741fa>

<https://www.pinecone.io/learn/series/image-search/zero-shot-image-classification-clip/>

Method 2, Prediction Using CLIP?? **Top k Cosine-Similarity**

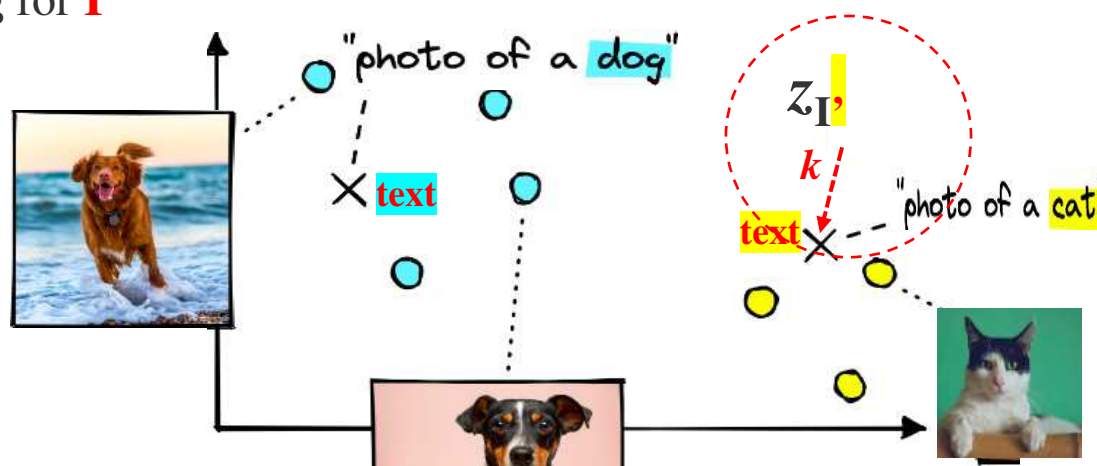
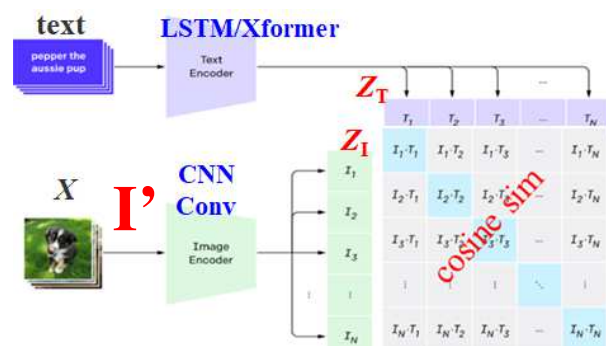
■ Create **text prompts as classes**

- Insert labels you want to classify into placeholder “??” in many text prompts
- **WHAT IF** no idea what classes to classify?
 - i.e. **NO** idea for proper text prompts



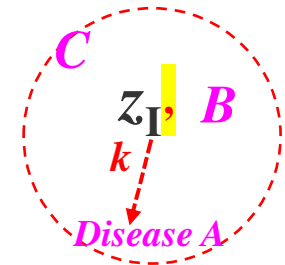
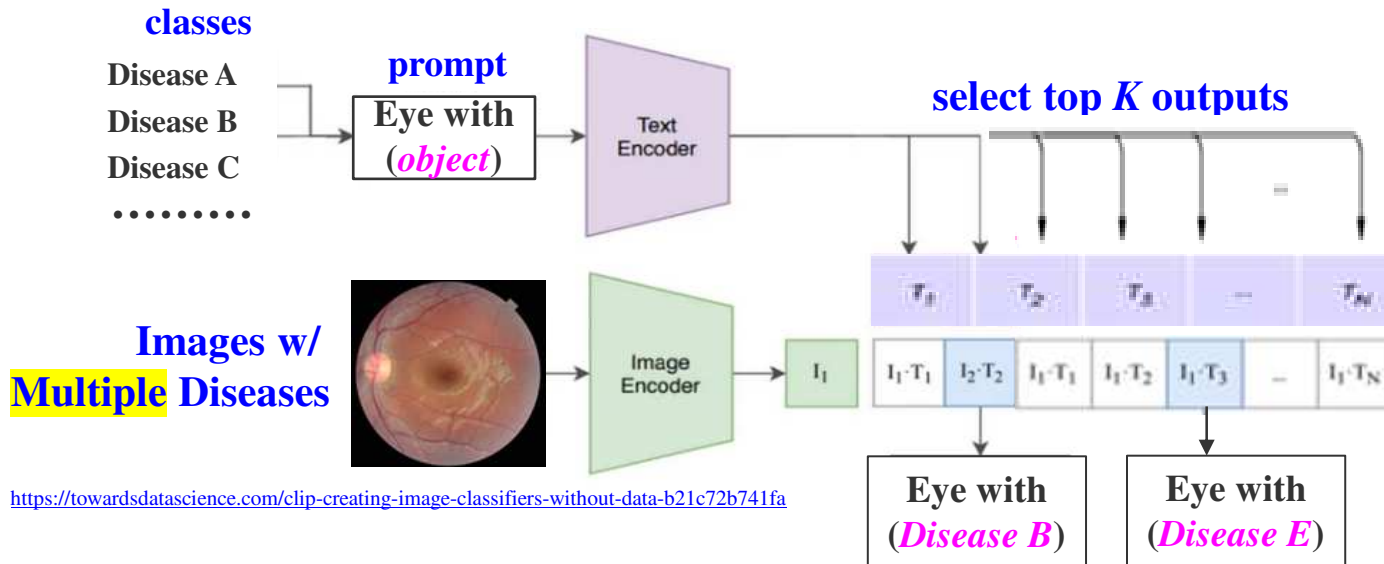
■ Method 2, **Top k Cosine-Similarity = k NN**

- Input a new image I' to the trained CNN to get $Z_{I'}$
- Use $Z_{I'}$ against **ALL** Z_T to find ones w/ **top k cos-sim**
- Now we find **multiple meaning** for I'



<https://www.pinecone.io/learn/series/image-search/zero-shot-object-detection-clip/>

How about Images with Multiple Meaning??



- Concept of **Zero-Shot** classification with top k search
= Multi-Label classification $\approx$ image captions
- See Appendix for **Multi-Label** classification
 - **Multiclass** vs. **Multilabel**

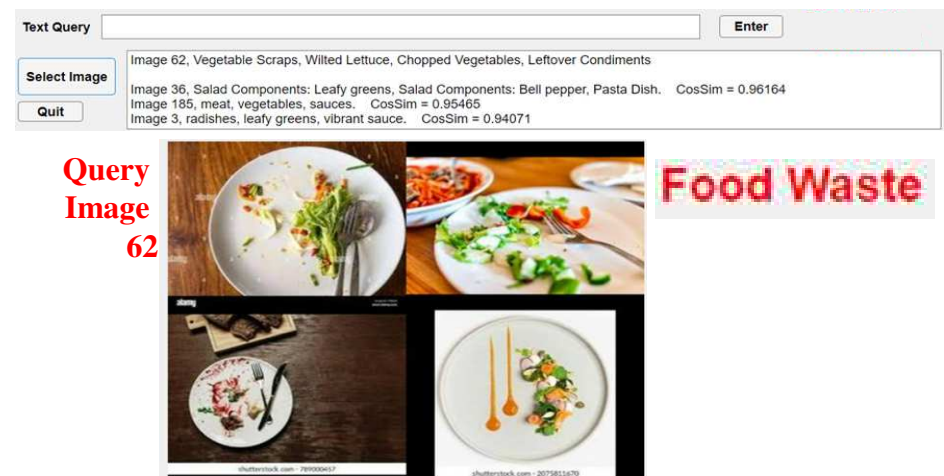
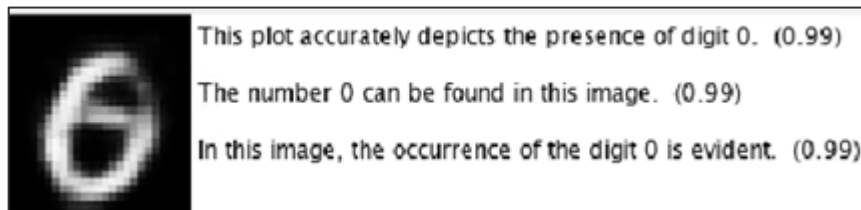
Images with Multiple Activities



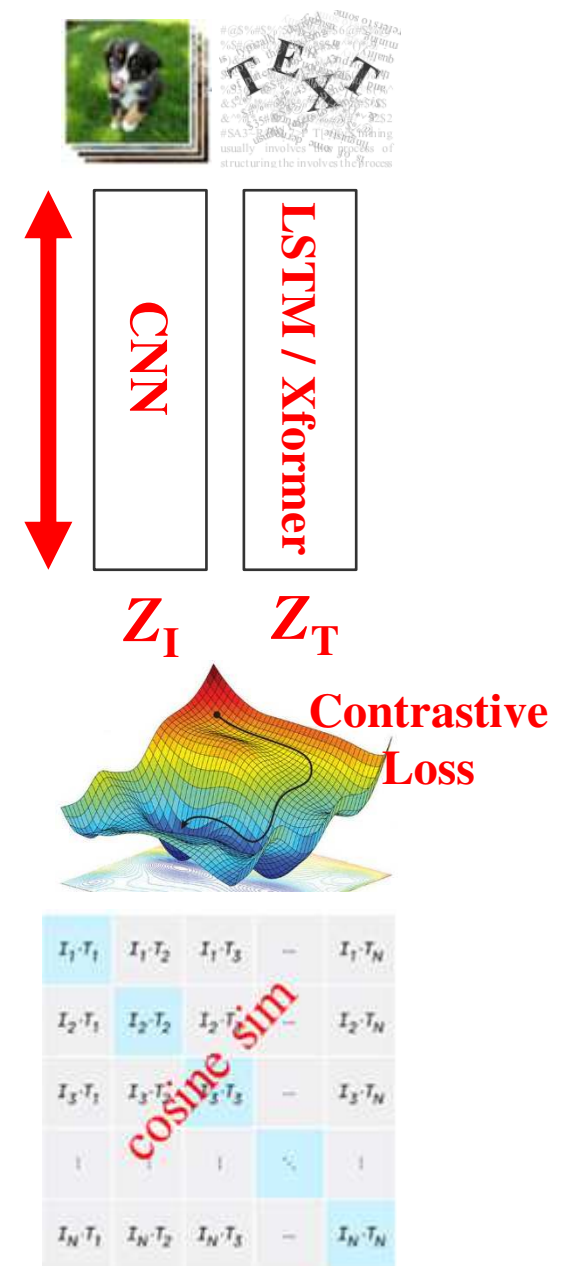
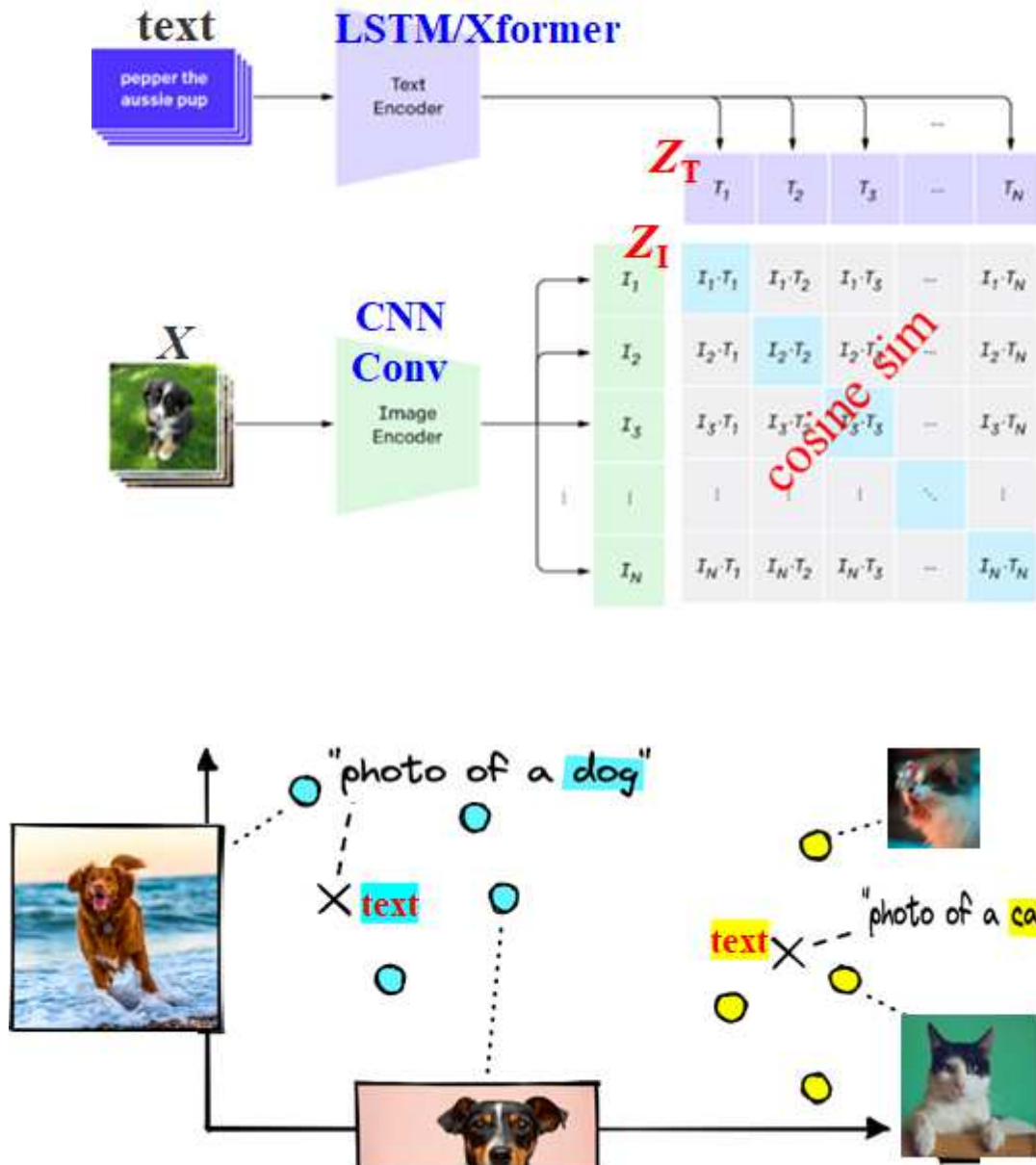
Outline

- Contrastive *L*anguage-*I*mage *P*re-training (**CLIP**)
- From problem to solution: training steps
- From problem to solution: more on prediction step
-  Experiments
- Contrastive loss, programs and pseudo code

Apply CLIP to walking results



Quick Reminder of Training CLIP



Why Not Just Similarity Search on Visual Embedding Z ?



Why not just compare cosine similarity among Z_I of all image??

Text Query Enter

Select Image

Quit

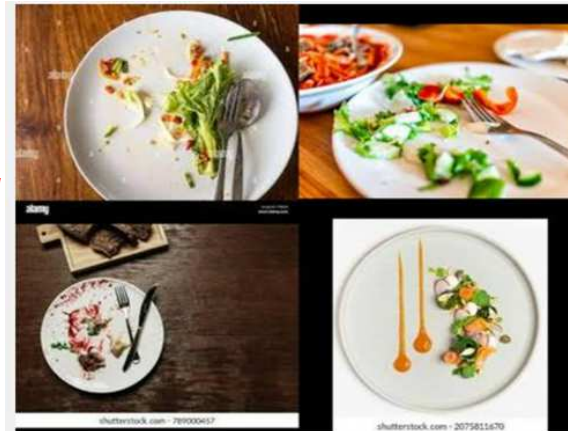
Image 62, Vegetable Scraps, Wilted Lettuce, Chopped Vegetables, Leftover Condiments

Image 36, Salad Components: Leafy greens, Salad Components: Bell pepper, Pasta Dish. CosSim = 0.96164

Image 185, meat, vegetables, sauces. CosSim = 0.95465

Image 3, radishes, leafy greens, vibrant sauce. CosSim = 0.94071

Query
Image
62



Food Waste

Few Pretrained CLIP Models

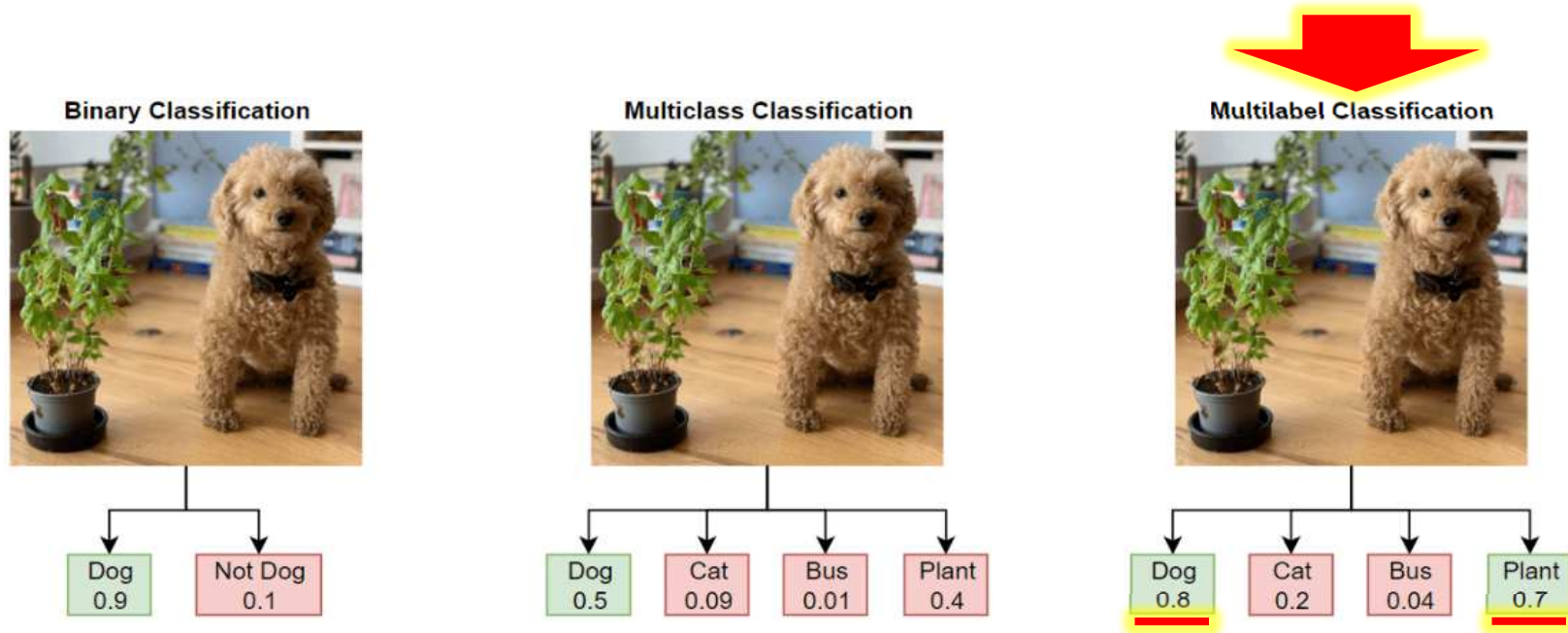
- Pretrained CLIP Models:
 - openai/clip-vit-base-patch32
 - openai/clip-vit-large-patch14
 - openai/clip-vit-base-patch16
 - openai/clip-vit-large-patch14-336
 - patrickjohncyh/fashion-clip
 - laion/CLIP-ViT-H-14-laion2B-s32B-b79K
 - laion/CLIP-ViT-B-32-laion2B-s34B-b79K
 - CIDAS/clipseg-rd64-refined
 - flaviagammarino/pubmed-clip-vit-base-patch32
 - laion/CLIP-ViT-L-14-DataComp.XL-s13B-b90K

- <https://medium.com/correll-lab/leveraging-pre-trained-clip-models-for-zero-shot-image-classification-ef61024f8c70>

Appendix

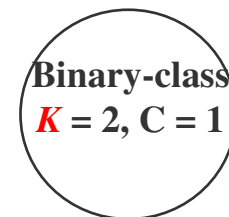
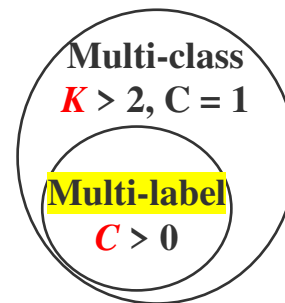
- Multiclass vs. Multilabel
- More CLIP Examples / References
- **Siamese** Network

Multilabel Image Classification Using Deep Learning



<https://www.mathworks.com/help/deeplearning/ug/multilabel-image-classification-using-deep-learning.html>

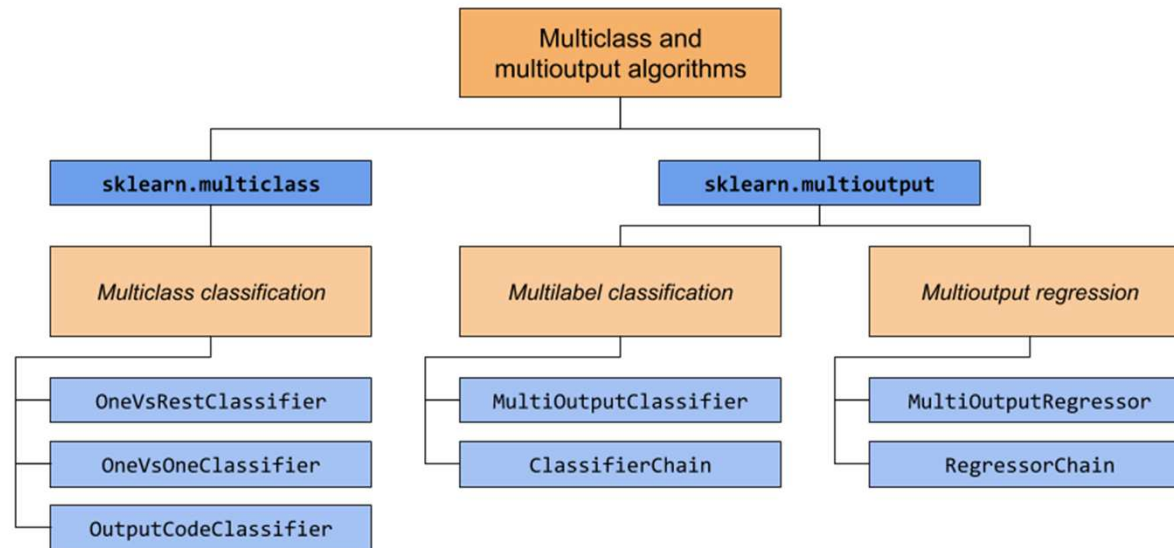
- **Multilabel** cross entropy
- $$= \frac{1}{L} \sum_{i=1}^L -Y_i \log(P_i)$$
 - $L = \text{multiple}$ classes i can belong to



K = total # of classes
 C = # of classes each record may belong

Multiclass vs. Multilabel

- <https://scikit-learn.org/1.5/modules/multiclass.html>



	Number of targets	Target cardinality	Valid <u>type_of_target</u>
Multiclass classification	1	>2	'multiclass'
Multilabel classification	>1	2 (0 or 1)	'multilabel-indicator'
Multiclass-multioutput classification	>1	>2	'multiclass-multioutput'
Multioutput regression	>1	Continuous	'continuous-multioutput'

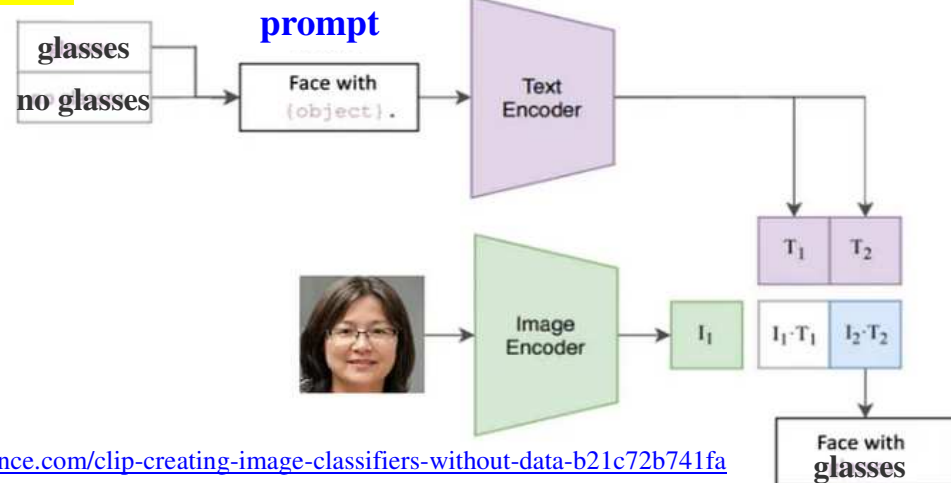
After Training, Then Prediction Using CLIP

- Create a classifier to classify images into classes / labels.....
 - But, training **without** explicit labels...

training data w/ text description
BUT, **NO** labels



NEW labels



<https://towardsdatascience.com/clip-creating-image-classifiers-without-data-b21c72b741fa>

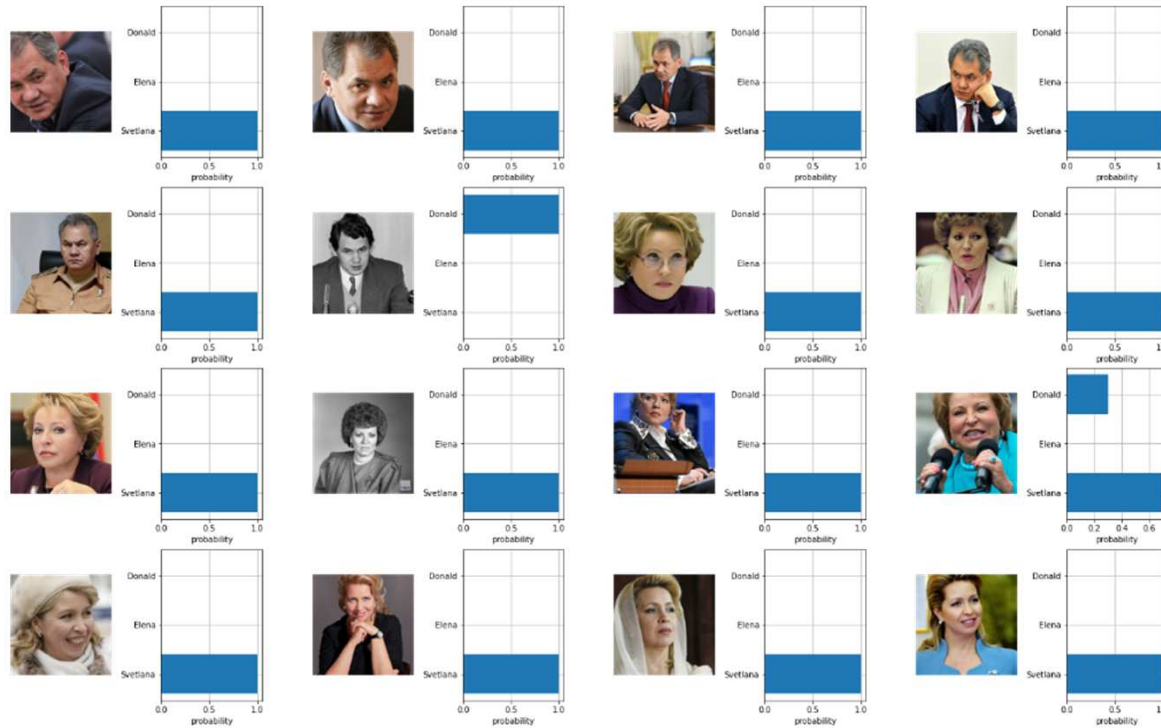
- Add text prompts w/ labels so text encoder learn more from text descriptions
 - But, different words in prompt can lead to different text **Z**, which can affect classifier

- ['no glasses', 'glasses',] — 0.82 accuracy
- ['face without glasses', 'face with glasses'] — 0.89 accuracy
- ['photo of a man with no glasses', 'photo of a man with glasses'] — 0.94

CLIP Colab Example

■ CLIP Colab example at

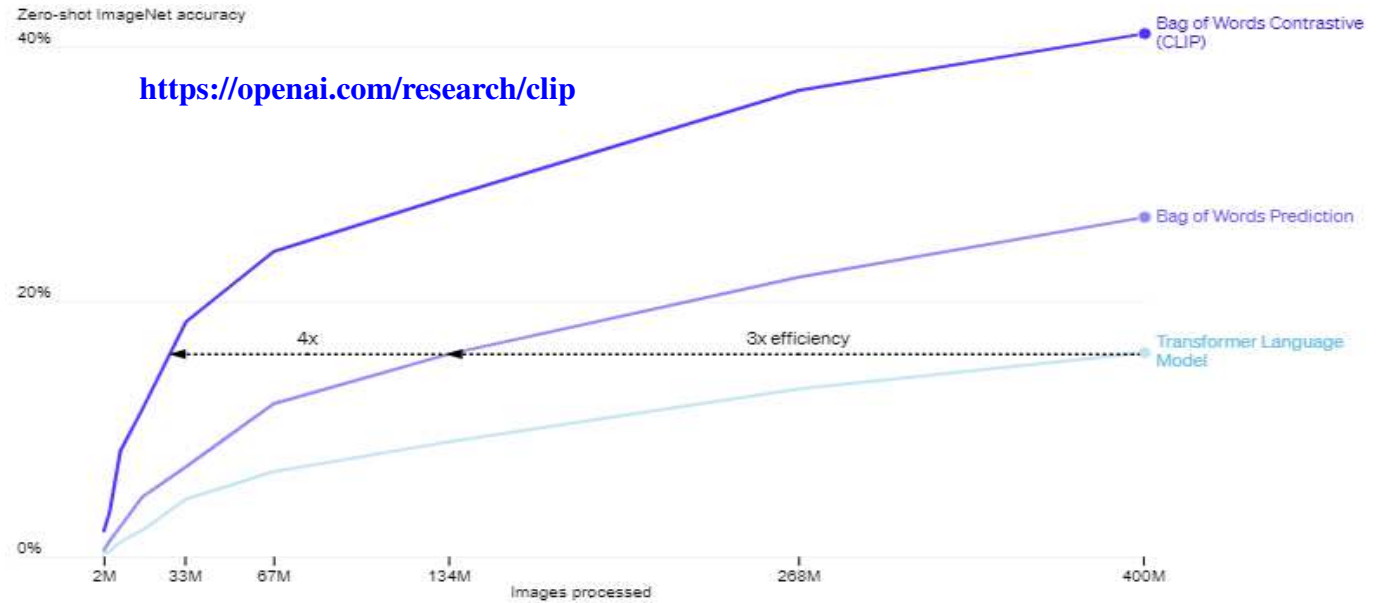
- <https://colab.research.google.com/drive/1ZSDOa6W5NxNdIwT79zFCEE4UwQ3PZjoU?usp=sharing>



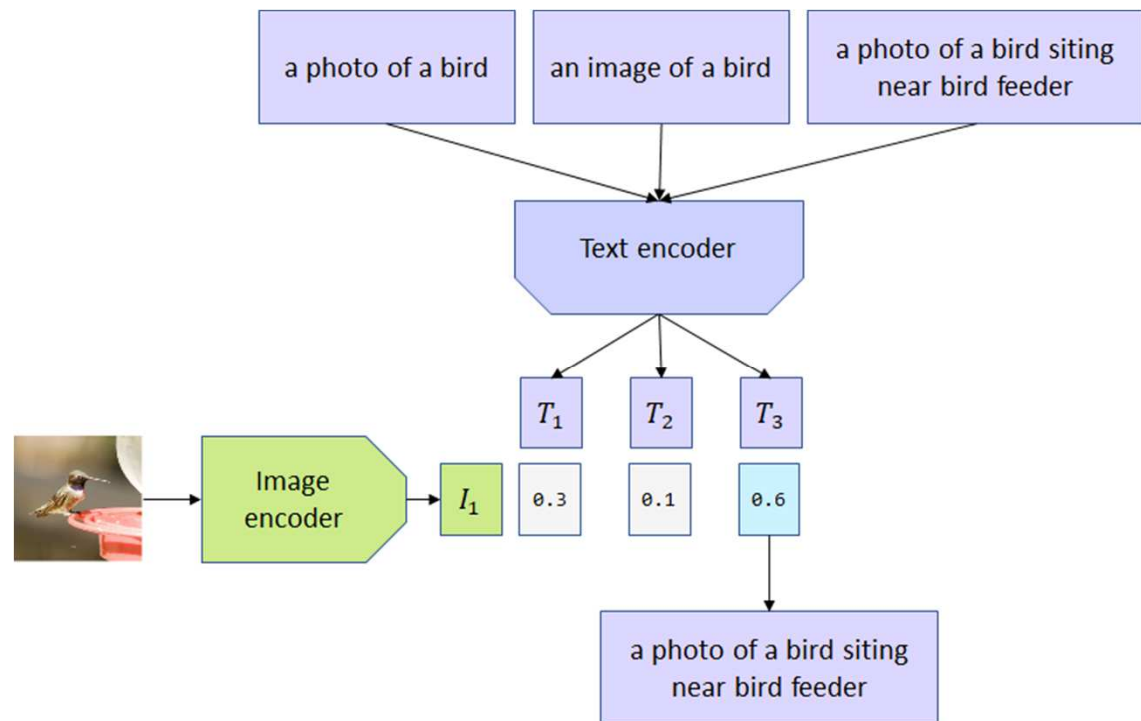
CLIP References

- CLIP from OpenAI: what is it and how you can try it out yourself
- <https://habr.com/en/articles/537334/>
 - <https://colab.research.google.com/drive/1ZSDOa6W5NxNdIwT79zFCEE4UwQ3PZjoU?usp=sharing#scrollTo=y2XRe5xHBiOi>
- **CLIP: Creating Image Classifiers Without Data**
- <https://towardsdatascience.com/clip-creating-image-classifiers-without-data-b21c72b741fa>
- **Natural language image search with a Dual Encoder**
 - https://keras.io/examples/vision/nl_image_search/

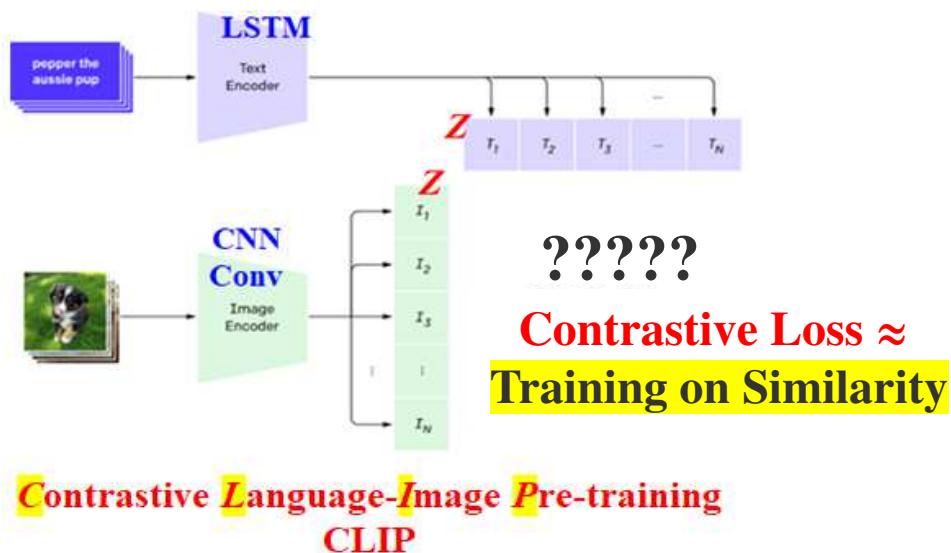
OpenAI CLIP Evaluation



CLIP Prediction



On A Side Track, **Siamese** Network, Similarity Detection



Target: similar
Predicted: similar
Score: 0.97809

Target: dissimilar
Predicted: dissimilar
Score: 0.0031511

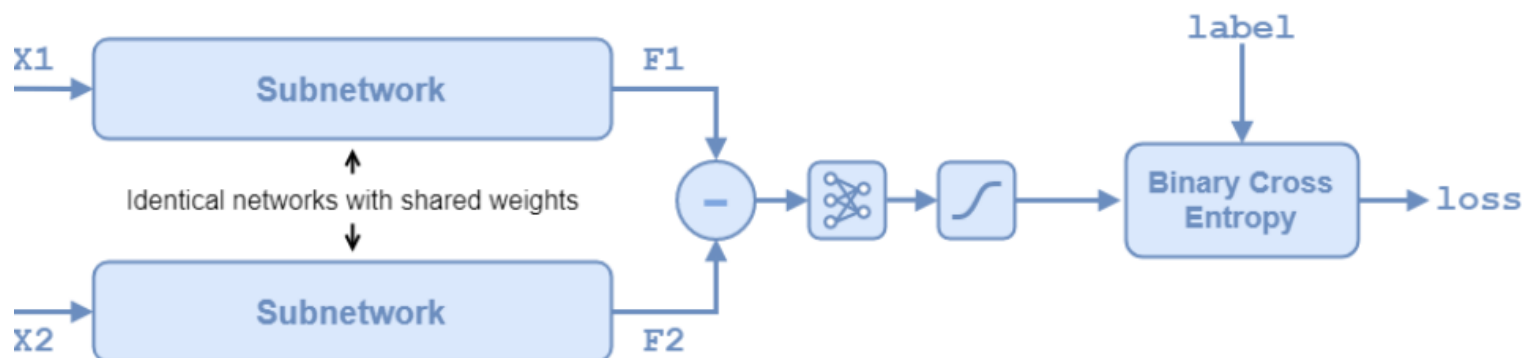
ਗ ਗ ਏ ਏ

Target: similar
Predicted: similar
Score: 0.60938

Target: similar
Predicted: similar
Score: 0.98135

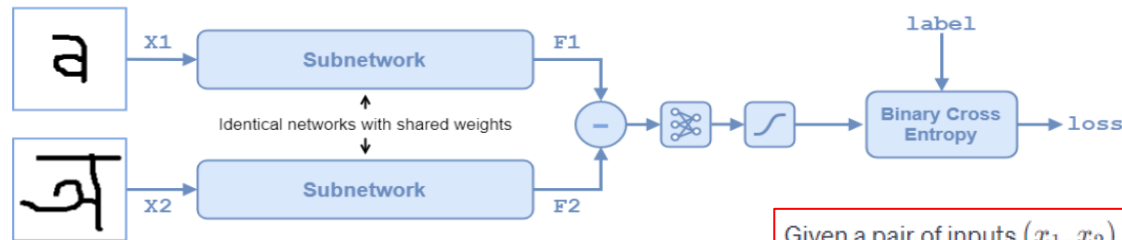
ਨ ਨ ਘ ਘ

- Compare signatures based on **learned** similarity



<https://www.mathworks.com/help/deeplearning/ug/train-twin-network-to-compare-images.html>

Siamese Network & Contrastive Loss Function



$$L(x_1, x_2, y) = \frac{1}{2} \cdot (y \cdot d^2 + (1 - y) \cdot \max(0, m - d)^2)$$

Given a pair of inputs (x_1, x_2) , where x_1 and x_2 are two instances to be compared, and y is the binary label indicating whether the instances are of the same class ($y = 1$) or different classes ($y = 0$), the contrastive loss is defined as:

$$L(x_1, x_2, y) = \frac{1}{2} \cdot (y \cdot d^2 + (1 - y) \cdot \max(0, m - d)^2)$$

where:

- d is the Euclidean distance or some other similarity measure between the embedded representations of x_1 and x_2 .
- m is a margin that is used to control the separation between positive and negative pairs. Pairs with a distance smaller than m (for positive pairs) or larger than m (for negative pairs) contribute to the loss.

The contrastive loss encourages the network to learn representations in which similar instances have small distances (d), while dissimilar instances have distances larger than the specified margin m . The factor y ensures that the loss is only applied to pairs of instances with the same class label ($y = 1$ for positive pairs) or different class labels ($y = 0$ for negative pairs).

In the context of Siamese Networks, the network is trained to minimize the overall contrastive loss over a batch of pairs, encouraging the model to learn embeddings that capture the similarities and differences between instances effectively.

Siamese Application

- Compare signatures

Target: dissimilar
Predicted: dissimilar
Score: 4.7014e-07

ਸ ਅ

Target: similar
Predicted: similar
Score: 0.9971

ਣ ਣ

Target: similar
Predicted: similar
Score: 0.97809

ਗ ਗ

Target: dissimilar
Predicted: dissimilar
Score: 0.0031511

ਏ ਓ

Target: dissimilar
Predicted: dissimilar
Score: 0.080849

ਸ਼ ਝ

Target: similar
Predicted: similar
Score: 0.97661

ੳ ੳ

Target: similar
Predicted: similar
Score: 0.60938

ਨ ਨ

Target: similar
Predicted: similar
Score: 0.98135

ਘ ਘ

<https://www.mathworks.com/help/deeplearning/ug/train-twin-network-to-compare-images.html>